

# Contents of Module 2

| Unit No. | Contents                                                                                                                                                                                                                                                                                                     | Hrs. |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
|          | <b>Combinational Circuits using basic gates as well as MSI devices</b>                                                                                                                                                                                                                                       |      |
| 2        | Arithmetic Circuits: Half adder, Full adder, Ripple carry adder, Carry Look ahead adder, Half Subtractor, Full Subtractor, multiplexer, cascading of Multiplexer , demultiplexer, decoder, Comparator (Multiplexer and demultiplexer gate level upto 4:1).<br>MSI devices: IC7483, IC74151, IC74138, IC7485. | 7    |
|          |                                                                                                                                                                                                                                                                                                              |      |

# Text Book for module 2:

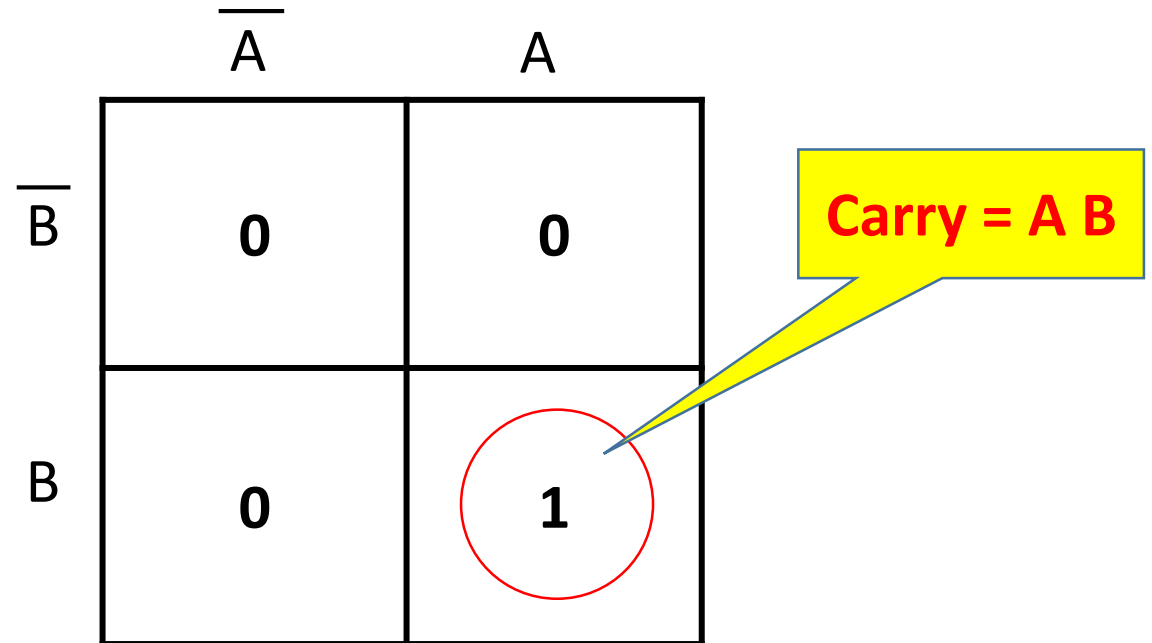
1

R. P. Jain, Modern Digital Electronics, Tata McGraw Hill Education, Third Edition 2003.

# Half Adder Circuit

Half adder is a logic circuit used to add two one bit binary numbers

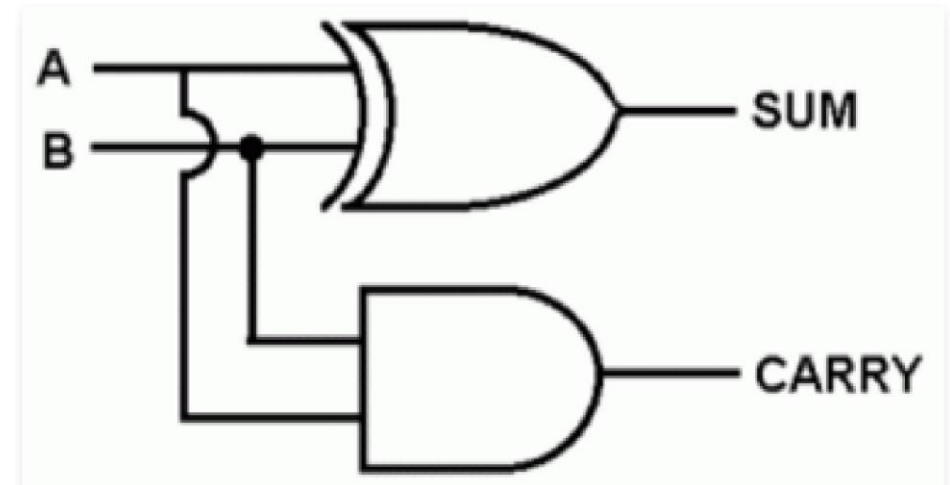
| INPUTS |   | OUTPUTS   |         |
|--------|---|-----------|---------|
| A      | B | CARRY (C) | SUM (S) |
| 0      | 0 | 0         | 0       |
| 0      | 1 | 0         | 1       |
| 1      | 0 | 0         | 1       |
| 1      | 1 | 1         | 0       |



|                |                |     |                 |
|----------------|----------------|-----|-----------------|
|                | $\overline{A}$ | $A$ |                 |
| $\overline{B}$ | 0              | 1   | $A\overline{B}$ |
| $B$            | 1              | 0   | $\overline{A}B$ |

$$\therefore \text{Sum} = \overline{A}B + A\overline{B}$$

**Circuit of HA**



# Full adder circuit

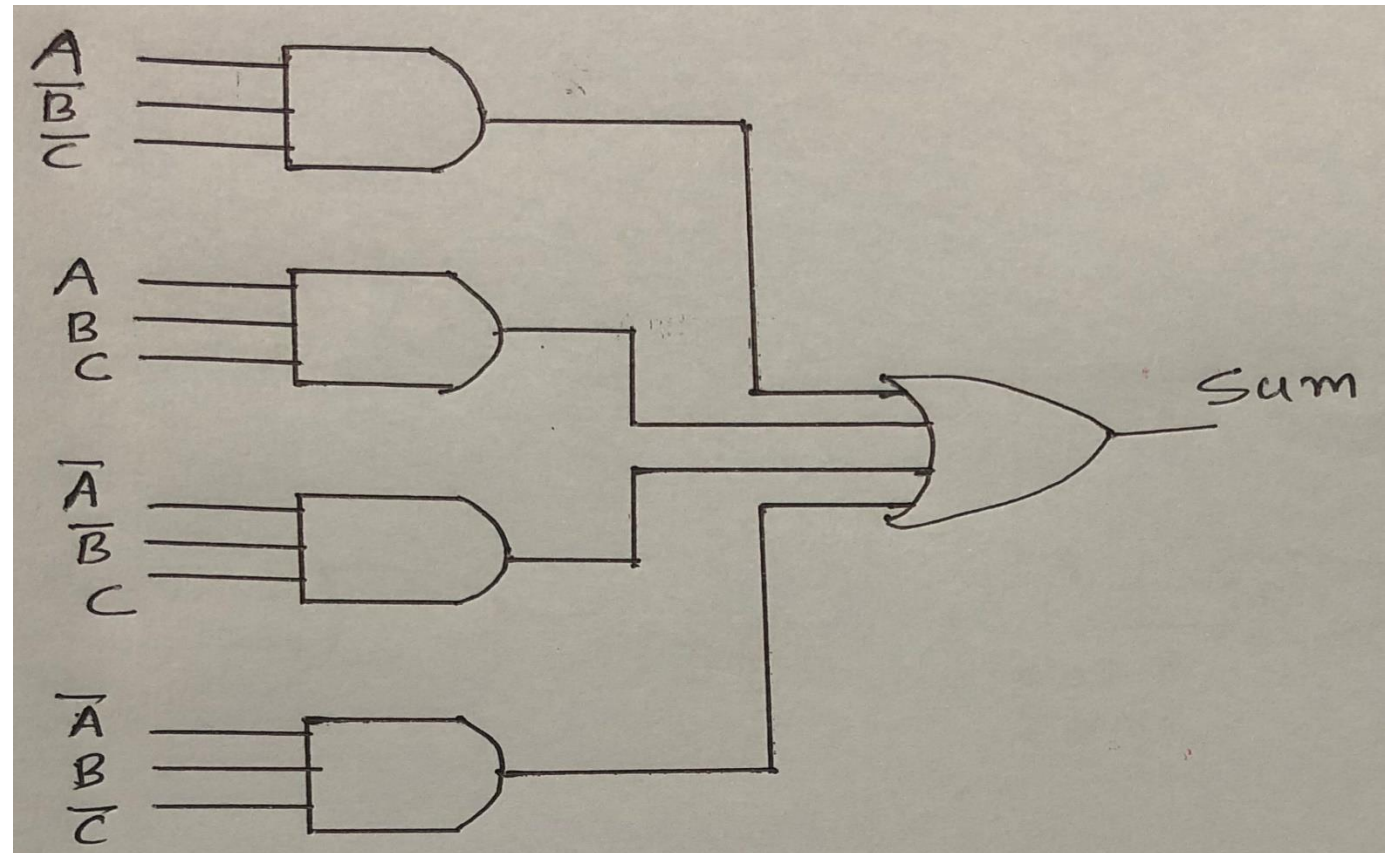
Full adder is a logic circuit used to add three one bit binary numbers

Truth Table of Full adder

| A | B | C <sub>in</sub> | C <sub>out</sub> | Sum |
|---|---|-----------------|------------------|-----|
| 0 | 0 | 0               | 0                | 0   |
| 0 | 0 | 1               | 0                | 1   |
| 0 | 1 | 0               | 0                | 1   |
| 0 | 1 | 1               | 1                | 0   |
| 1 | 0 | 0               | 0                | 1   |
| 1 | 0 | 1               | 1                | 0   |
| 1 | 1 | 0               | 1                | 0   |
| 1 | 1 | 1               | 1                | 1   |

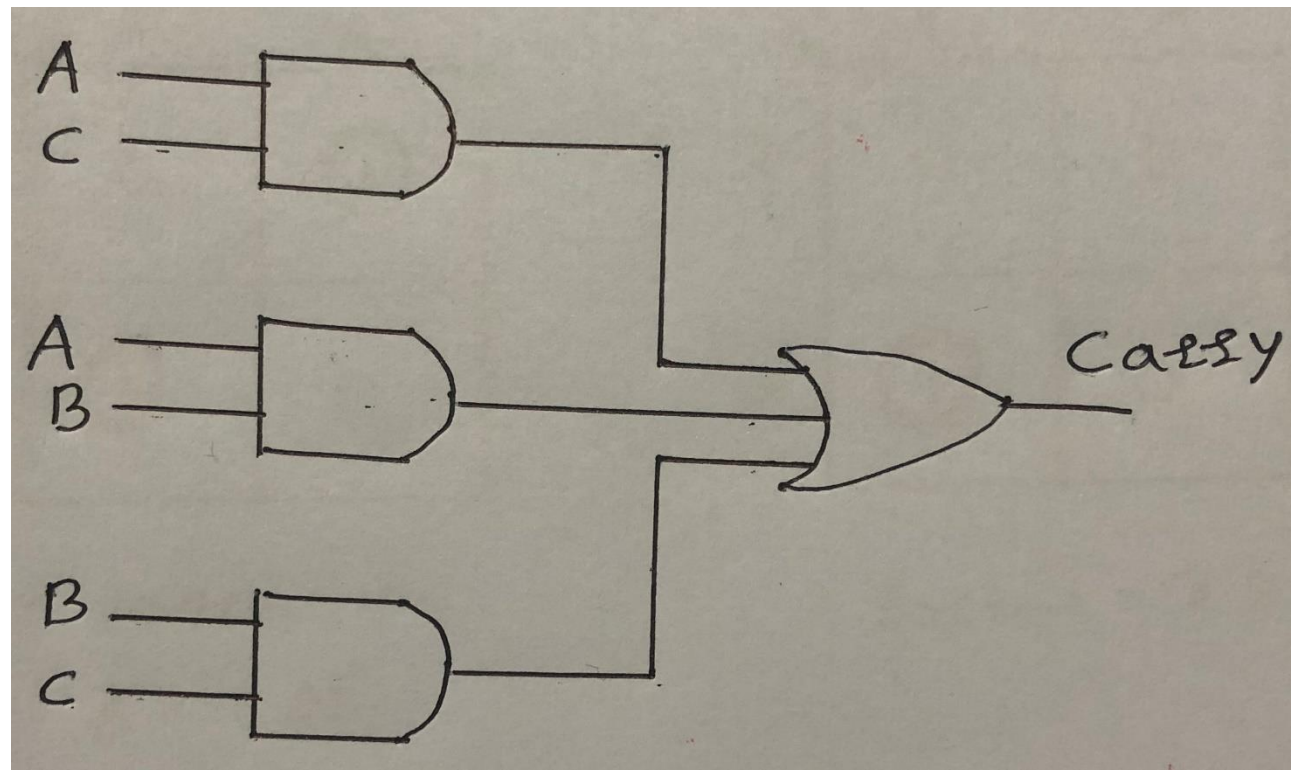
|                             | $\overline{A}$ | $A$ |
|-----------------------------|----------------|-----|
| $\overline{B} \overline{C}$ | 0              | 1   |
| $\overline{B} C$            | 1              | 0   |
| $B \overline{C}$            | 0              | 1   |
| $B C$                       | 1              | 0   |

$$\text{Sum} = \overline{A} \overline{B} \overline{C} + \overline{A} B \overline{C} + A \overline{B} C + A B C$$

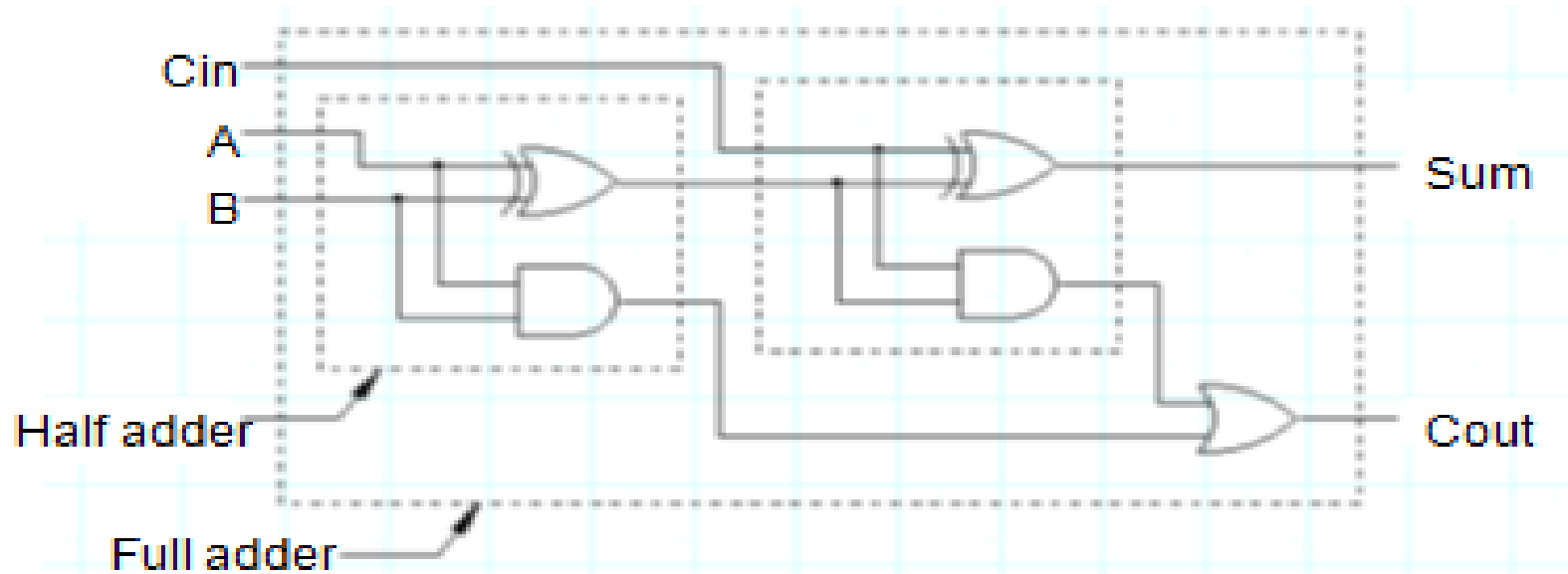


|                             | $\overline{A}$ | $A$ |
|-----------------------------|----------------|-----|
| $\overline{B} \overline{C}$ | 0              | 0   |
| $\overline{B} C$            | 0              | 1   |
| $B C$                       | 1              | 1   |
| $B \overline{C}$            | 0              | 1   |

$$Carry = AC + AB + BC$$



# Logic diagram of Full adder using 2 Half adders)



|                             | $\overline{A}$ | $A$ |
|-----------------------------|----------------|-----|
| $\overline{B} \overline{C}$ | 0              | 1   |
| $\overline{B} C$            | 1              | 0   |
| $B \overline{C}$            | 0              | 1   |
| $B C$                       | 1              | 0   |

$$\begin{aligned}
 \text{Sum} &= \overline{A} \overline{B} \overline{C} + A B C + \overline{A} \overline{B} C + \overline{A} B \overline{C} \\
 &= A(\overline{B} \overline{C} + B C) + \overline{A}(\overline{B} C + B \overline{C}) \\
 &= A(B \odot C) + \overline{A}(B \oplus C) \\
 &= A \oplus B \oplus C
 \end{aligned}$$

|                             | $\overline{A}$ | $A$ |
|-----------------------------|----------------|-----|
| $\overline{B} \overline{C}$ | 0              | 0   |
| $\overline{B} C$            | 0              | 1   |
| $B C$                       | 1              | 1   |
| $B \overline{C}$            | 0              | 1   |

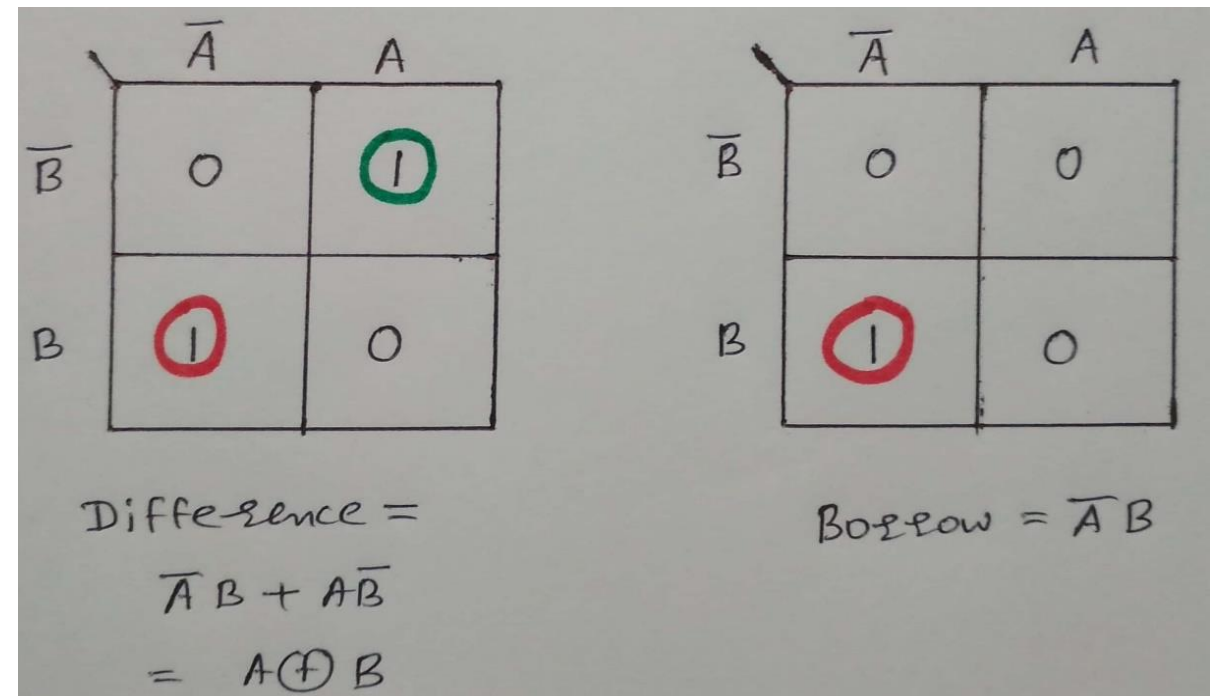
$$\begin{aligned}
 \text{Carry} &= \overline{A} B C + A \overline{B} C + A B \\
 &= C (\overline{A} B + A \overline{B}) + A B
 \end{aligned}$$

# Half subtractor

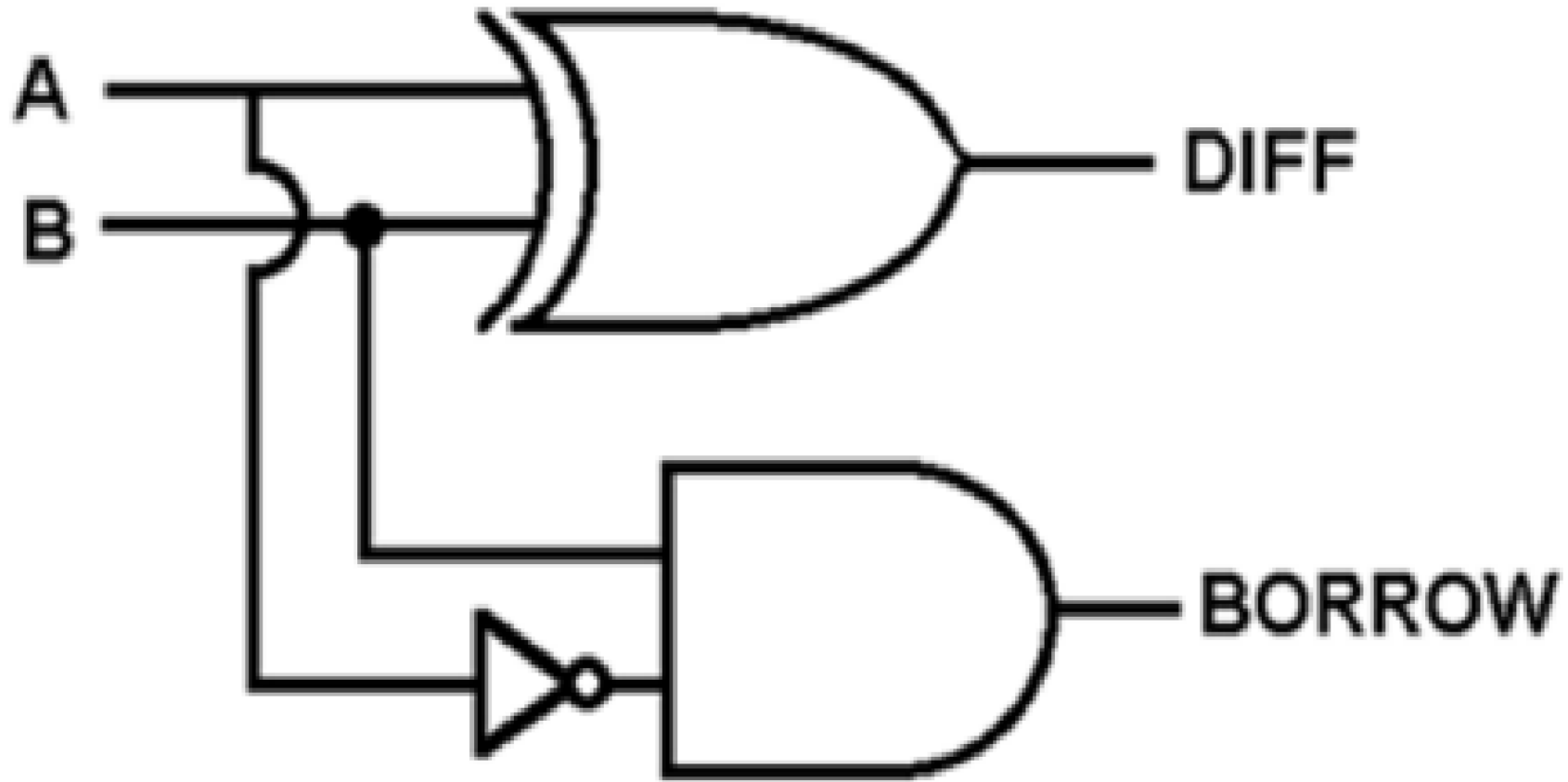
Half subtractor is a logic circuit used to subtract two one bit binary numbers

Truth table of Half subtractor

| A | B | Borrow | Difference |
|---|---|--------|------------|
| 0 | 0 | 0      | 0          |
| 0 | 1 | 1      | 1          |
| 1 | 0 | 0      | 1          |
| 1 | 1 | 0      | 0          |



# Logic diagram of Half Subtractor



# Full Subtractor

Truth table of Full subtractor

| A | B | C | Borrow | Difference |
|---|---|---|--------|------------|
| 0 | 0 | 0 | 0      | 0          |
| 0 | 0 | 1 | 1      | 1          |
| 0 | 1 | 0 | 1      | 1          |
| 0 | 1 | 1 | 1      | 0          |
| 1 | 0 | 0 | 0      | 1          |
| 1 | 0 | 1 | 0      | 0          |
| 1 | 1 | 0 | 0      | 0          |
| 1 | 1 | 1 | 1      | 1          |

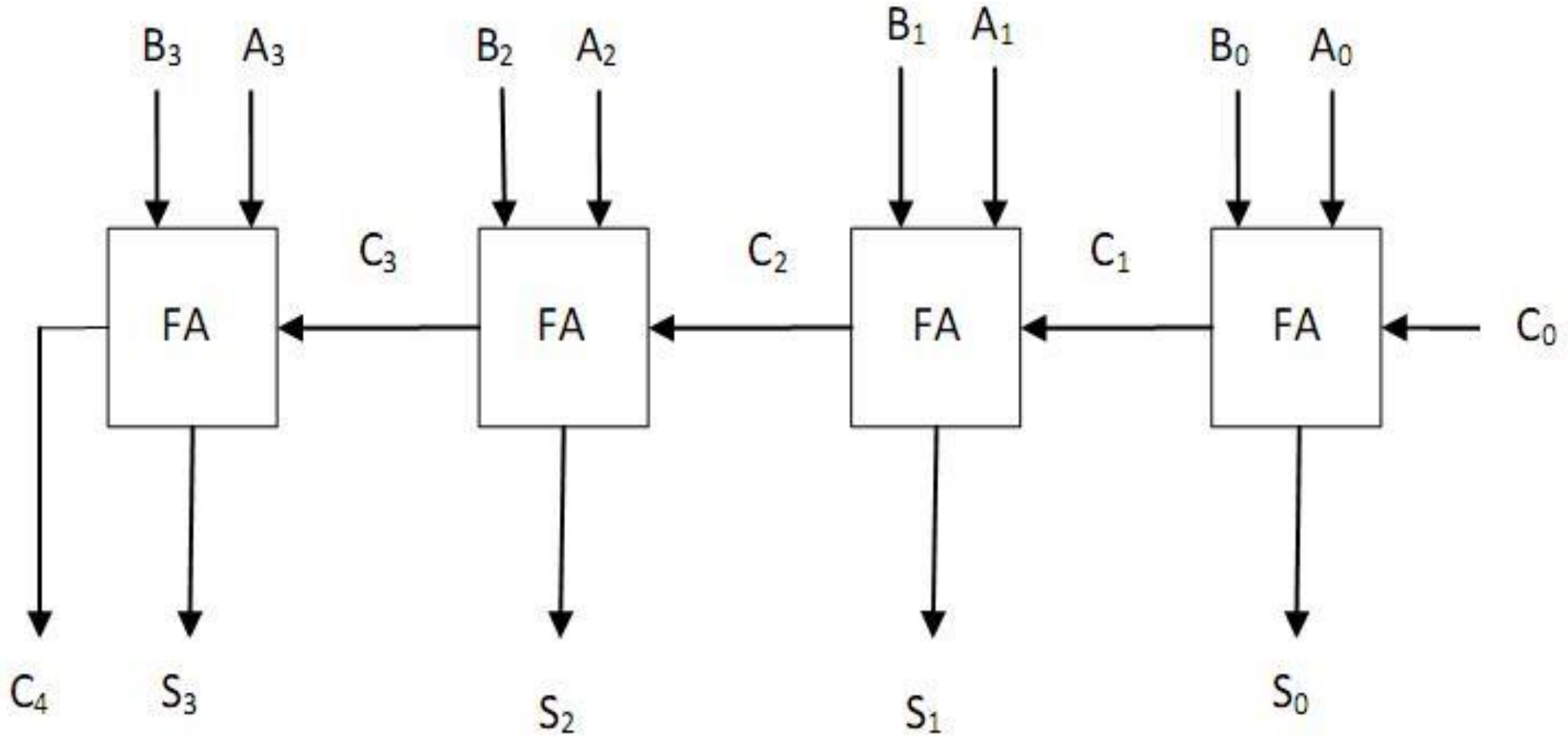
# Ripple Carry Adder (RCA)

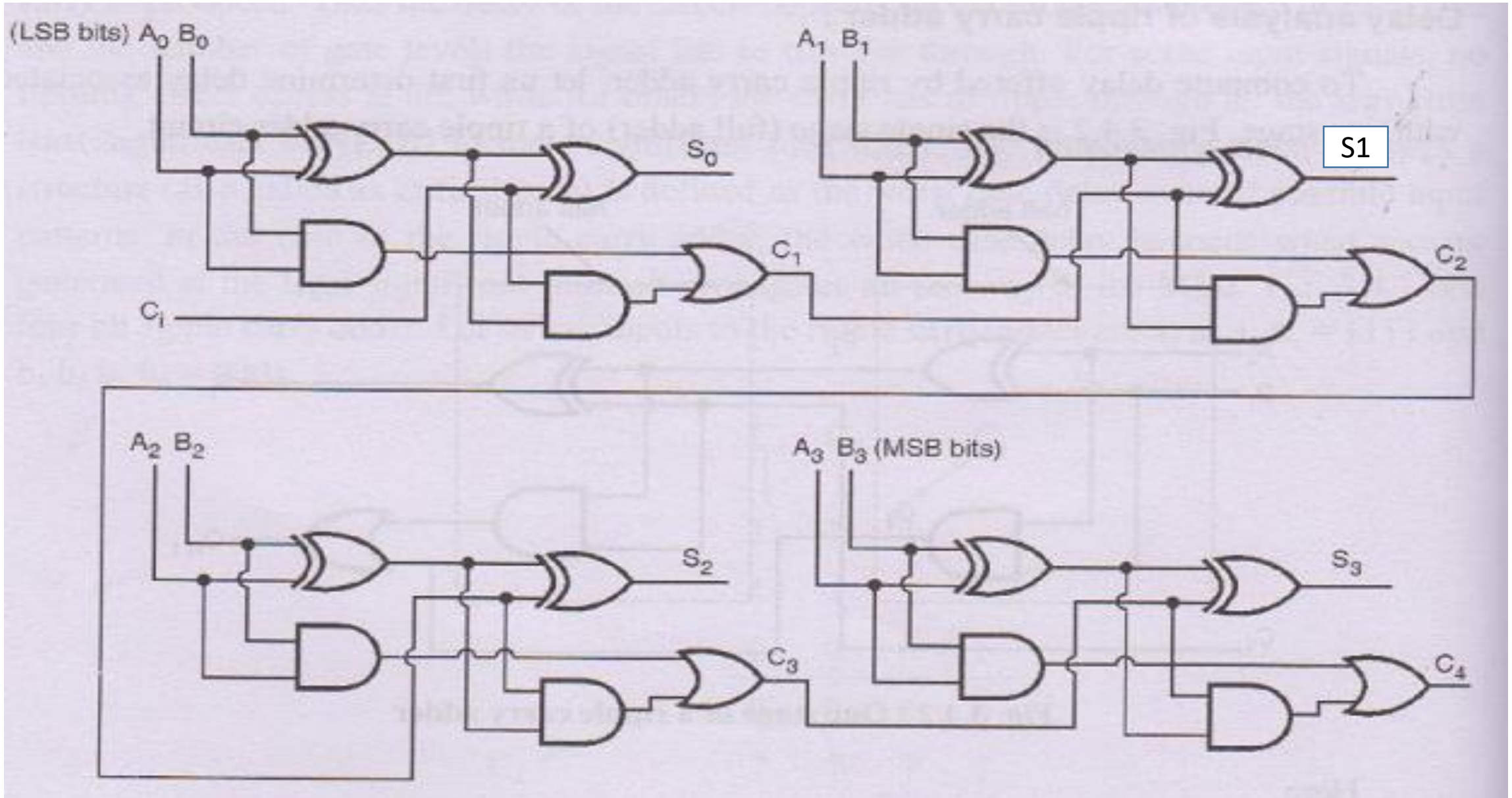
A binary adder is a digital circuit that produces the arithmetic sum of two binary numbers. It can be constructed with full adder connected in cascade, the output carry from each full adder connected to the input carry of the next full adder in the chain. Fig shows the interconnection of four full adder (FA) circuits to provide a **4-bit binary Ripple Carry Adder**. The augend bits of **A** and addend bits of **B** are designated by subscript numbers from right to left, with subscript 0 denoting the least significant bit. The carries are connected in the chain through the full adders. The input carry to the adder is  $C_0$  and it ripples through the full adder to the output carry  $C_4$ . The S output generate the required sum bits. An **n**-bit adder requires **n** full adders with each output connected to the input carry of the next higher order full adder.

# Ripple Carry Adder (RCA)

**The carry signal ripples through the full adder stages; hence it is called as Ripple Carry adder.**

# Four bit Ripple Carry Adder





# Disadvantage of RCA

- It is slow (Delay is more)
- In RCA the final output will be available after  $(2n + 1)$  gate delays
- In 4 bit RCA the final output will be available after  $(2n + 1)$  gate delays i.e. after 9 gate delays. [ $(2n + 1)$  gate delays for  $C_4$  and  $2n$  gate delays for  $S_3$ ].

| Sum O/P | Gate delays<br>required for sum<br>O/P = $2n$ | Carry O/P | Gate delays<br>required for carry<br>O/P = $(2n+1)$ |
|---------|-----------------------------------------------|-----------|-----------------------------------------------------|
| S0      | 2                                             | C1        | 3                                                   |
| S1      | 4                                             | C2        | 5                                                   |
| S2      | 6                                             | C3        | 7                                                   |
| S3      | 8                                             | C4        | 9                                                   |
| S4      | 10                                            | C5        | 11                                                  |
| S5      | 12                                            | C6        | 13                                                  |
| S6      | 14                                            | C7        | 15                                                  |
| S7      | 16                                            | C8        | 17                                                  |

# Carry Look-ahead Adder (CLA)

**Advantage:**  
**CLA is faster than RCA**

## CLA $\rightarrow$ Carry Lookahead Adder

Output carry of each Full Adder is written in terms of  $G$  (generate carry) and  $P$  (propagate carry)

| Inputs |   |       | Carry | Sum | $G = AB$ | $P = A \oplus B$ | $PC_i$ | $Co = G + PC_i$ |
|--------|---|-------|-------|-----|----------|------------------|--------|-----------------|
| A      | B | $C_i$ | $C_o$ | S   |          |                  |        |                 |
| 0      | 0 | 0     | 0     | 0   | 0        | 0                | 0      | 0               |
| 0      | 0 | 1     | 0     | 1   | 0        | 0                | 0      | 0               |
| 0      | 1 | 0     | 0     | 1   | 0        | 1                | 0      | 0               |
| 0      | 1 | 1     | 1     | 0   | 0        | 1                | 1      | 1               |
| 1      | 0 | 0     | 0     | 1   | 0        | 1                | 0      | 0               |
| 1      | 0 | 1     | 1     | 0   | 0        | 1                | 1      | 1               |
| 1      | 1 | 0     | 1     | 0   | 1        | 0                | 0      | 1               |
| 1      | 1 | 1     | 1     | 1   | 1        | 0                | 0      | 1               |

Logical operations

$$C_1 = G_0 + P_0 C_i$$

$$C_2 = G_1 + P_1 C_1$$

$$= G_1 + P_1 [G_0 + P_0 C_i]$$

$$= G_1 + P_1 G_0 + P_1 P_0 C_i$$

$$C_2 = G_1 + P_1 C_1$$

$$= G_1 + P_1 [G_0 + P_0 C_i]$$

$$= G_1 + P_1 G_0 + P_1 P_0 C_i$$

$$C_3 = G_2 + P_2 C_2$$

$$= G_2 + P_2 [G_1 + P_1 G_0 + P_1 P_0 C_i]$$

$$= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_i$$

$$C_3 = G_2 + P_2 C_2$$

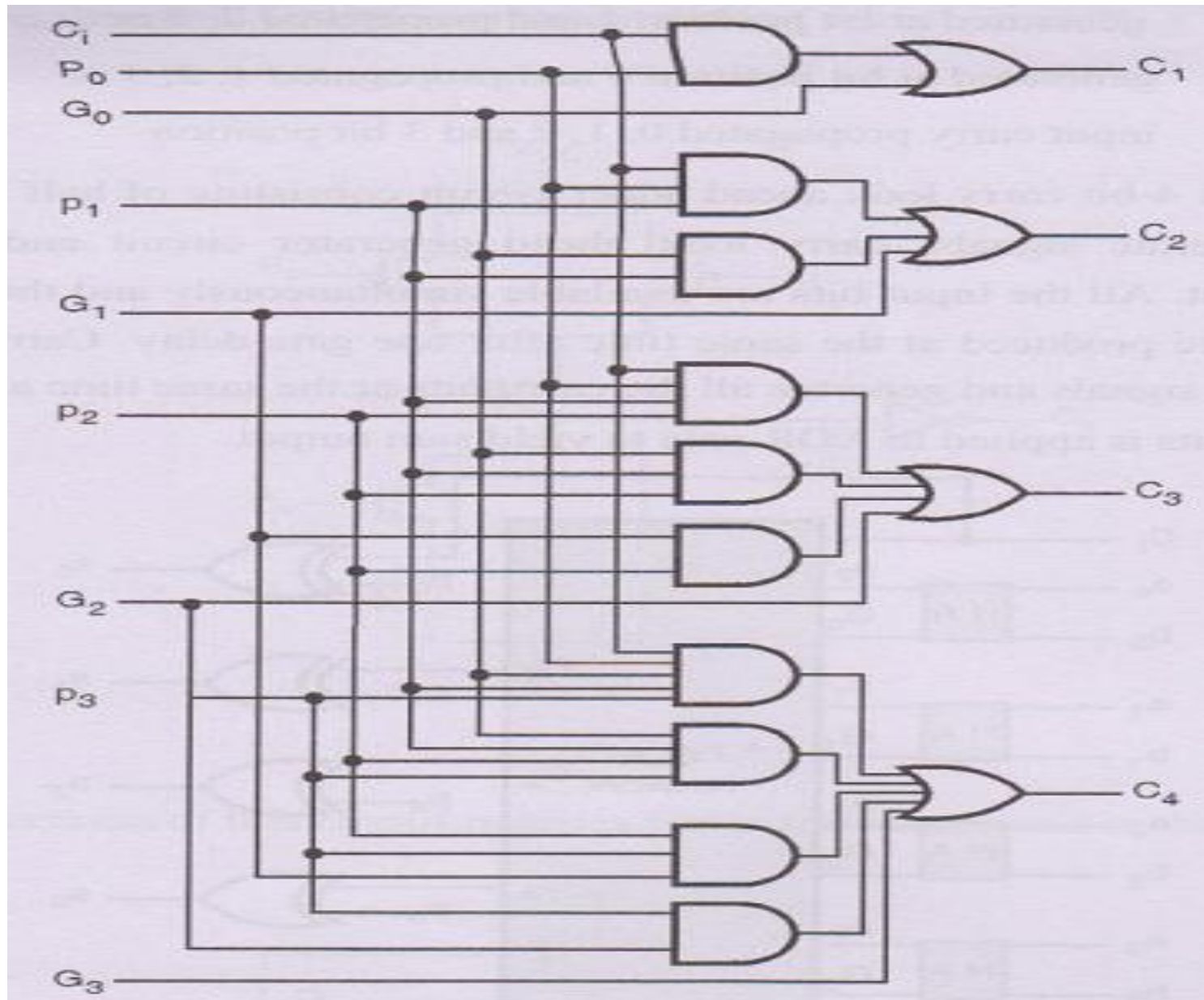
$$= G_2 + P_2 [G_1 + P_1 G_0 + P_1 P_0 C_i]$$

$$= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_i$$

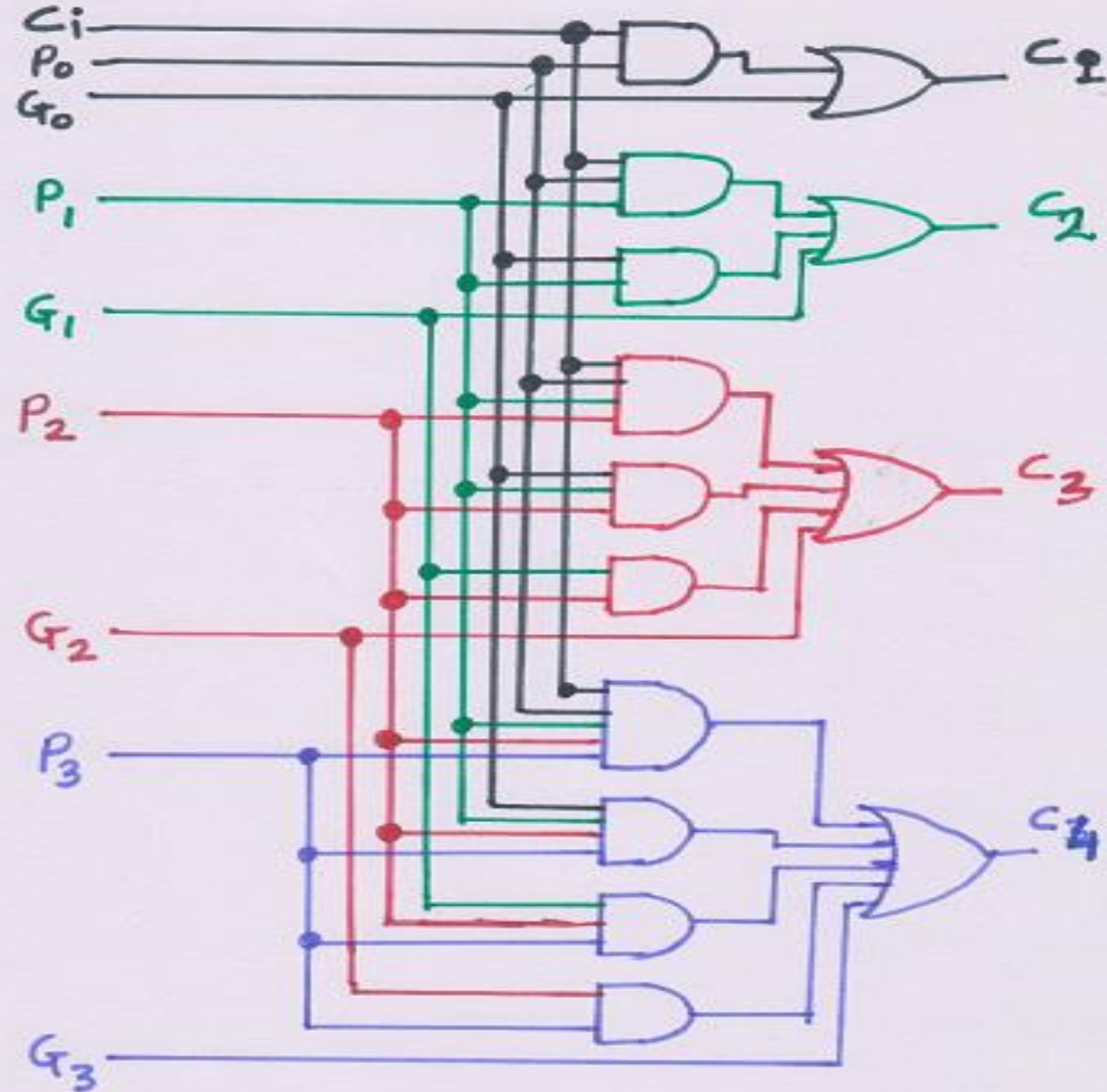
$$C_4 = G_3 + P_3 C_3$$

$$= G_3 + P_3 [G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_i]$$

$$= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_i$$

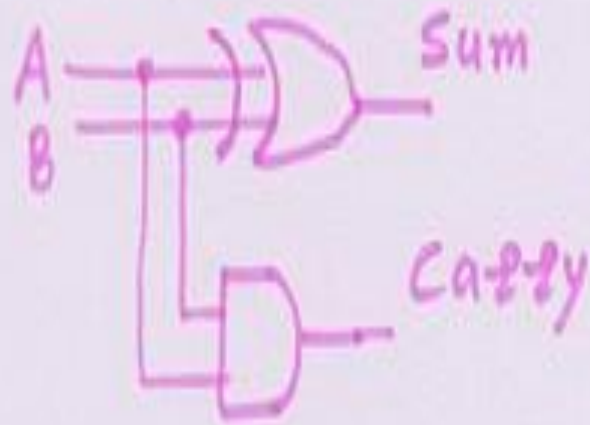


## Carry Look-Ahead generator ckt



- \* In  $n$  bit RCA the final o/p will be available after  $(2n+1)$  gate delays i.e. after 9 gate delays,  $[(2n+1) \text{ for } C_4 \text{ and } 2n \text{ for } S_3]$
- \* In CLA all carry o/p's  $C_4, C_3, C_2, C_1$  are available after 3 gate delays.
- \* In CLA  $S_0$  will be available after 2 gate delays and higher sum o/p's  $S_3, S_2, S_1$  are available after 4 gate delays.

## HA CKt



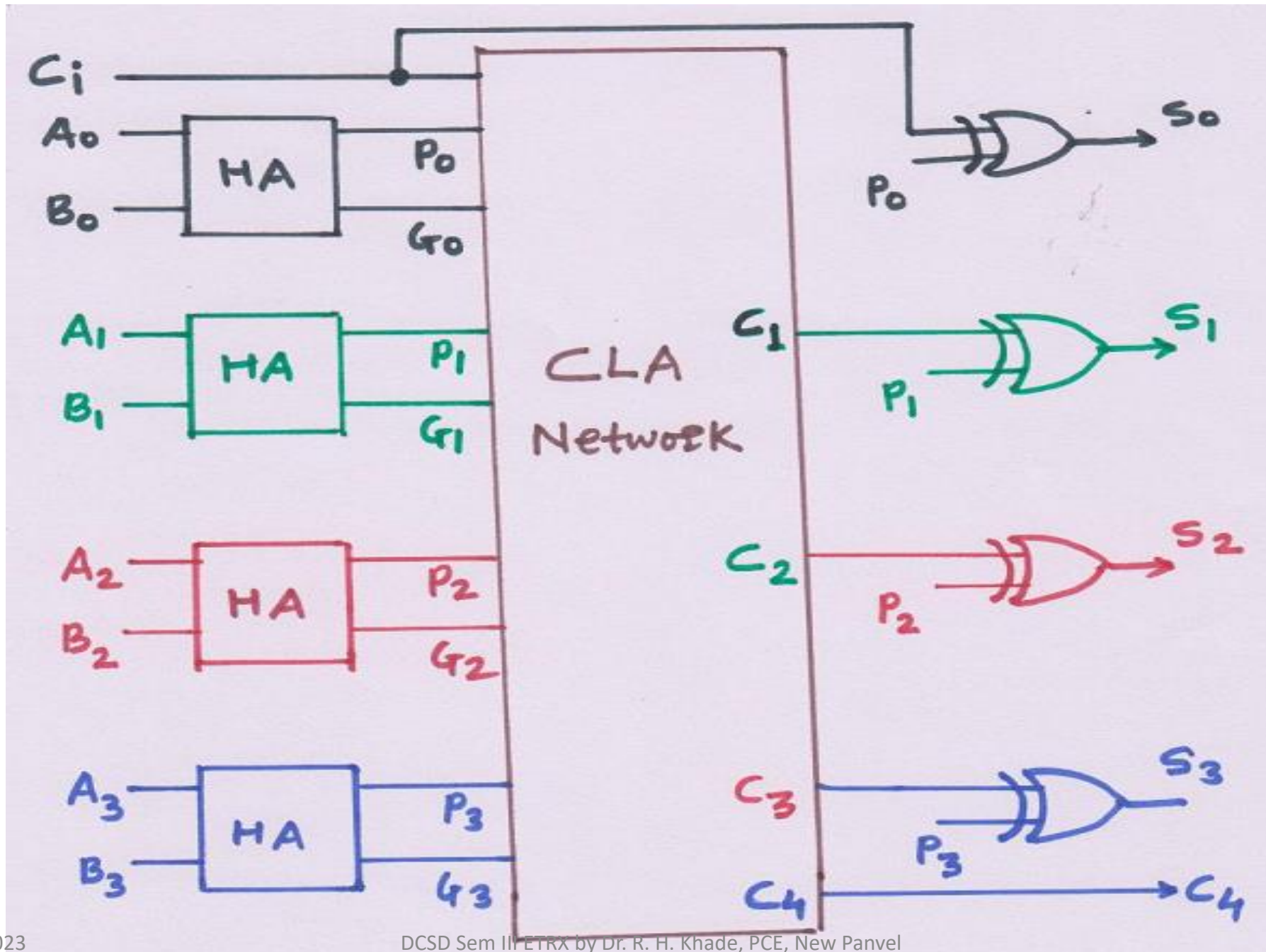
$$G = AB \text{ and } P = A \oplus B$$

$\therefore$  Half Adder CKt  
can be used to  
to produce  $G, P$

## Carry Look Ahead (CLA) Adder

4 bit CLA adder circuit consists of:

- ① Four HA to produce propagate and generate signals.
- ② Carry Look Ahead generator ckt.
- ③ Four XOR gates that produce sum outputs.

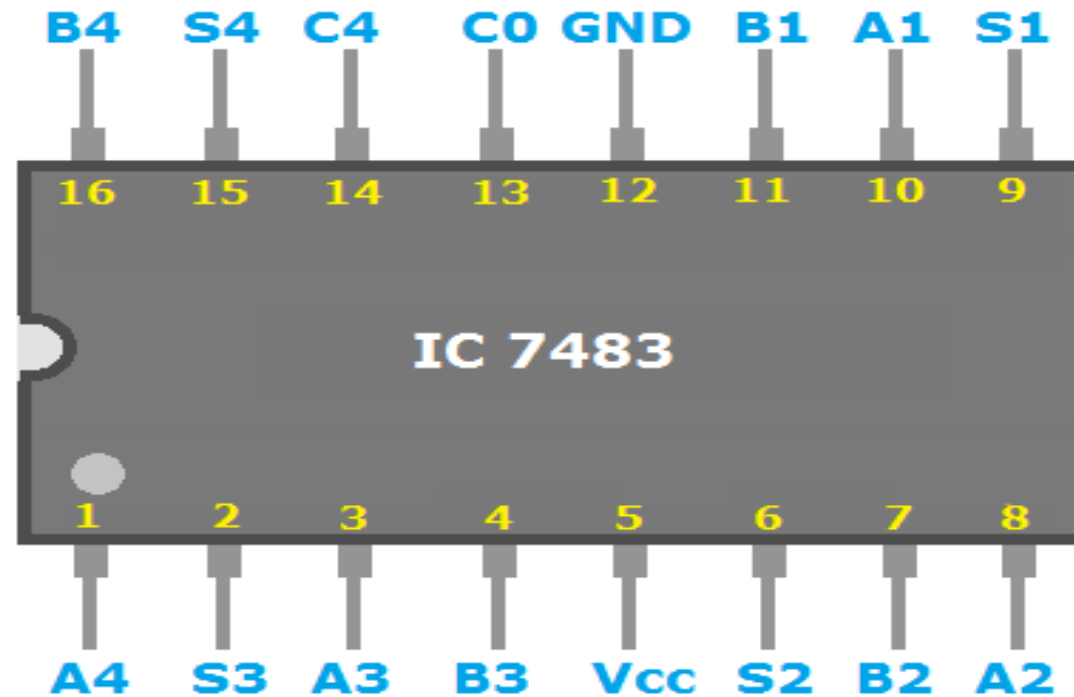


## Comparison of 4 bit RCA and CLA adders

| Parameter         | RCA                              | CLA adder                        |
|-------------------|----------------------------------|----------------------------------|
| $S_3$             | After $(2n)T_p$<br>i.e. $8T_p$   | After $4T_p$                     |
| $C_4$             | After $(2n+1)T_p$<br>i.e. $9T_p$ | After $3T_p$                     |
| No. of Gates      | Less                             | More<br>with more fan in         |
| Speed             | Slow                             | Fast                             |
| Area Required     | Less                             | More                             |
| Power dissipation | Less                             | More as<br>no. of gates are more |

# 4 Bit CLA Adder IC 7483

IC 7483 is 4 bit parallel adder which consists of 4 interconnected full adders along with Look Ahead Carry circuit.



IC 7483 Pin Diagram

# IC 7483 is 8 bit parallel adder using 7483 ICs

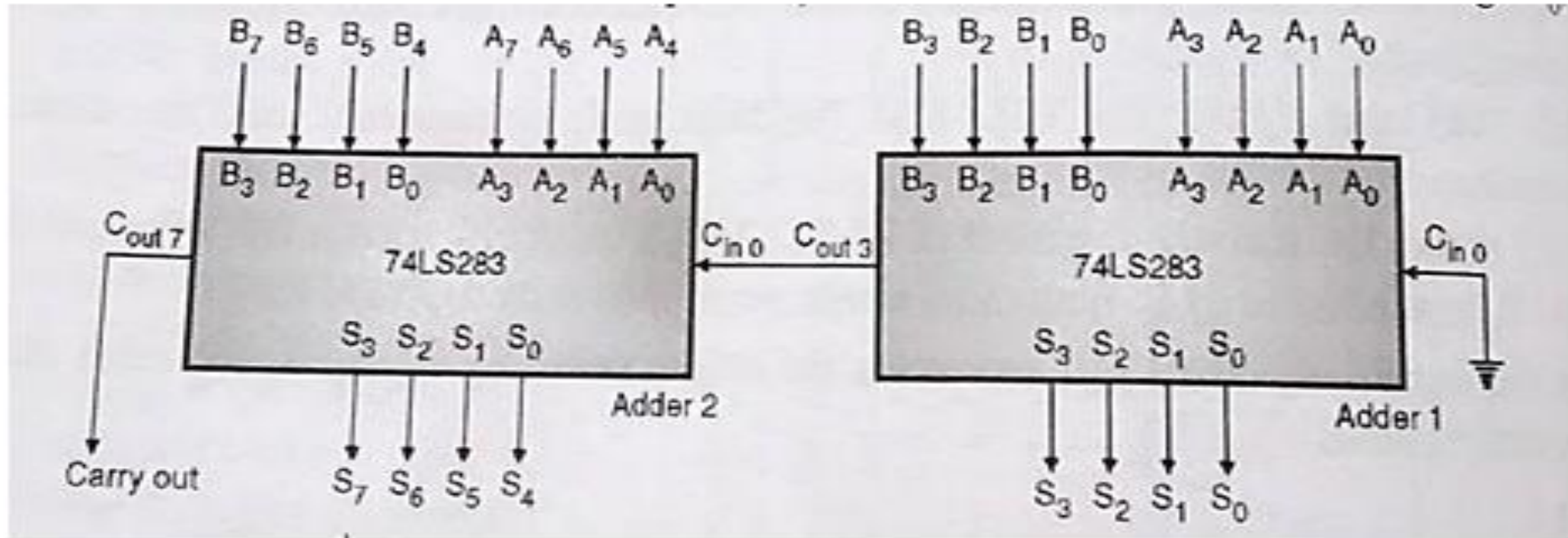
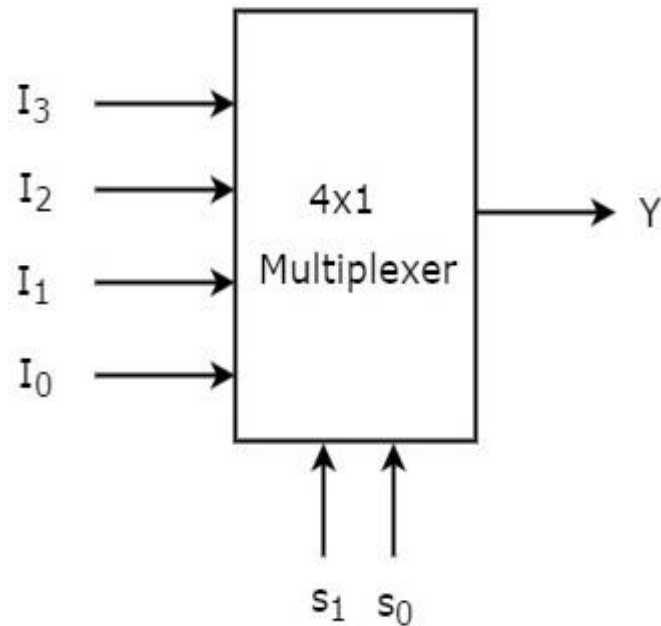


Fig2. Cascading of two IC 7483s

- IC 7483 is a 4 bit parallel adder which consists of four interconnected full adders along with the look ahead carry circuit. The pin diagram of IC 7483 is shown in fig. It is a 16pin IC. The inputs to the IC are A, B and  $C_{in0}$  while outputs are S and  $C_4$ .
- $A_3A_2A_1A_0$  is a 4 bit input word 'A' and  $B_3B_2B_1B_0$  is the second 4 bit input word 'B'.  $C_{in0}$  is the input carry. The IC adds the two four bit words along with input carry to produce a 4 bit sum and a one bit carry-out.  $C_{out3}$  represents the output carry.  $S_3, S_2, S_1, S_0$  represents sum output with  $S_3$  as the MSB.
- In order to design an 8 bit adder, we require two IC 7483s cascaded as shown in the figure.
- Adder-1 is the LSB adder and it adds the four LSB bits of the two 8-bit input words ie  $A_3-A_0$  and  $B_3-B_0$ . The carry input of first adder is supposed to be 0. Hence the carry in pin of LSB IC is connected to the ground. So the first IC adds the LSB bits of A and B, and produces  $S_3-S_0$  that is, LSB of sum, along with a carry out  $C_{out3}$ .
- This  $C_{out3}$  of adder-1 is connected to  $C_{in0}$  input of Adder-2. The second adder adds this carry and the four MSB bits of numbers  $A_7-A_4$  and  $B_7-B_4$  to produce MSB sum  $S_7-S_4$  along with final carry out bit  $C_{out7}$ .
- Thus adder-1 and adder-2 when cascaded as shown in the figure can add two 8-bit words.  $C_{out7}$  of adder-2 acts as the final output carry and the sum output is from  $S_7$  though  $S_0$ .

# Multiplexer

- Multiplexer is a combinational circuit which has many inputs and only one output. Data from only one input is transmitted to output at a time.



4 : 1 Multiplexer has four data inputs  $I_3$ ,  $I_2$ ,  $I_1$  &  $I_0$ , two selection lines  $s_1$  &  $s_0$  and one output  $Y$ . The **block diagram** of 4x1 Multiplexer is shown in the adjacent figure.

2 : 1 multiplexer will have two input lines and one output line. It will have one select lines.

8 : 1 multiplexer will have eight input lines and one output line. It will have three select lines.

16 : 1 multiplexer will have sixteen input lines and one output line. It will have four select lines.

# Truth table of 4x1 Multiplexer

| Select Inputs |            | Output |
|---------------|------------|--------|
| $S_1$ or A    | $S_0$ or B | Y      |
| 0             | 0          | $I_0$  |
| 0             | 1          | $I_1$  |
| 1             | 0          | $I_2$  |
| 1             | 1          | $I_3$  |

In case of mux, If **N** are number of select lines then no of input lines =  $2^N$

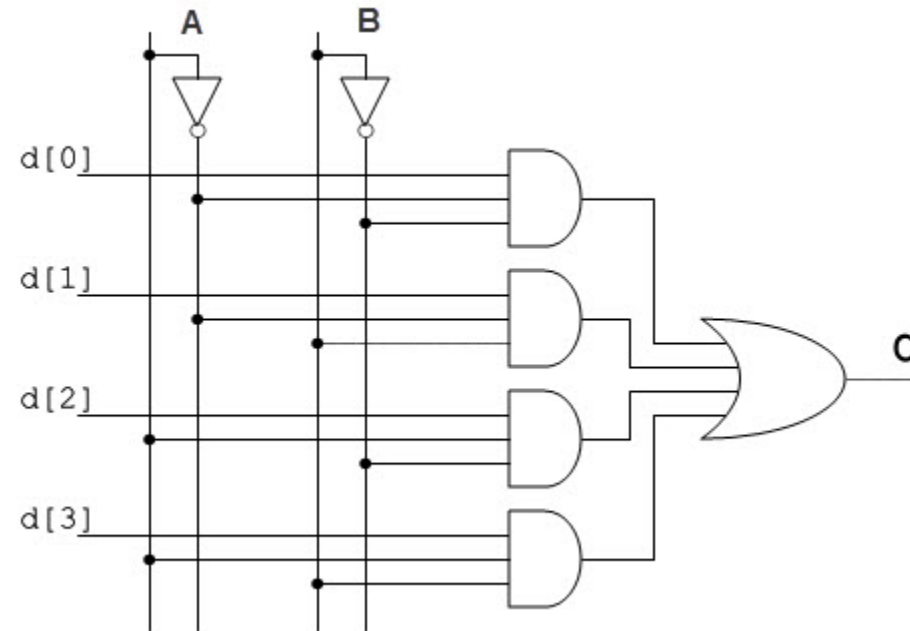
From Truth table, we can directly write the **Boolean function** for output, Y as:

$$Y = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3$$

OR

$$Y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$

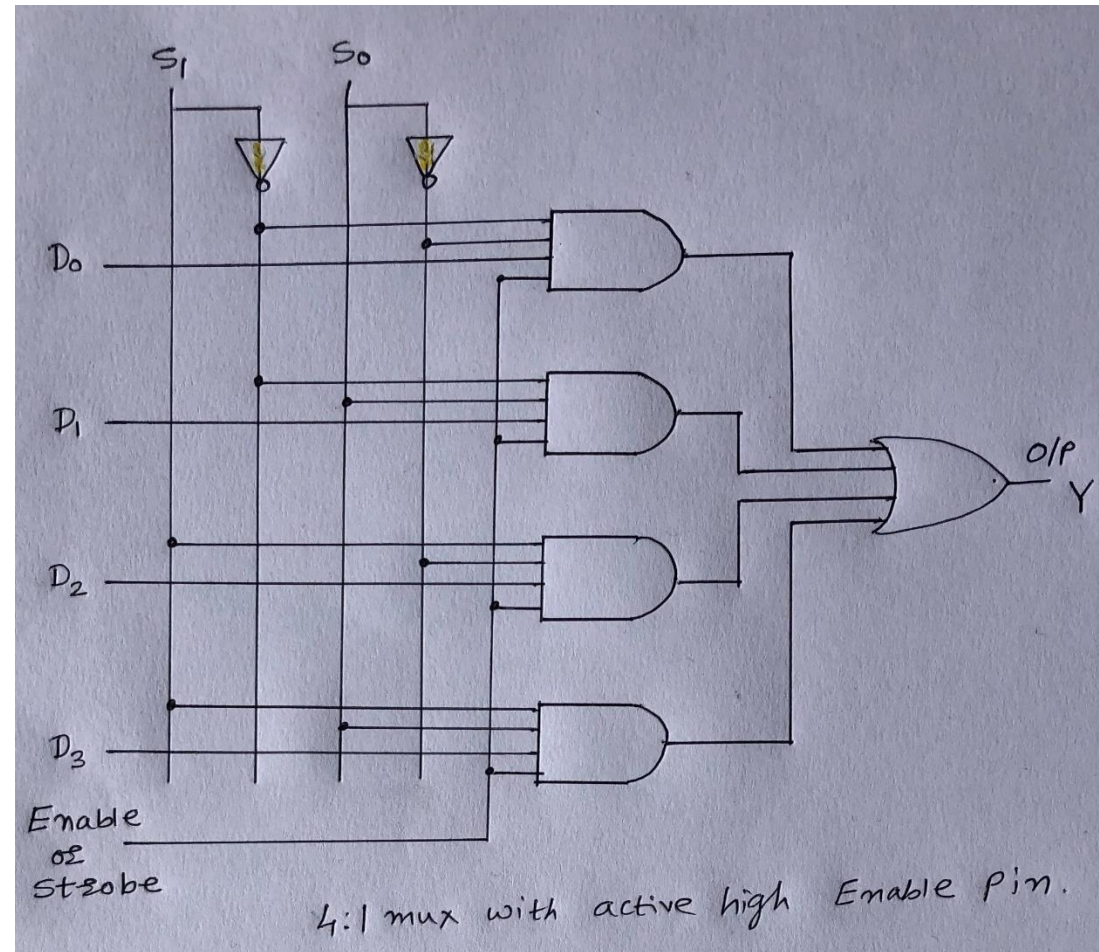
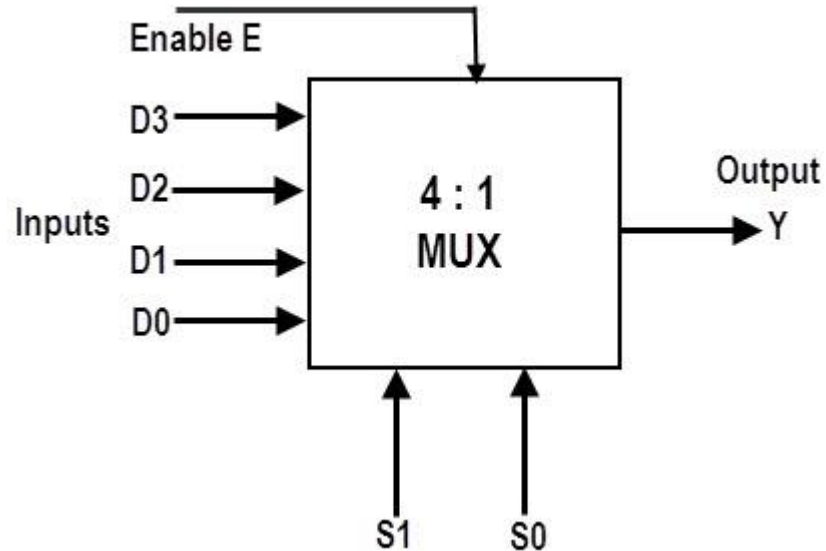
# Logic diagram of 4 : 1 multiplexer



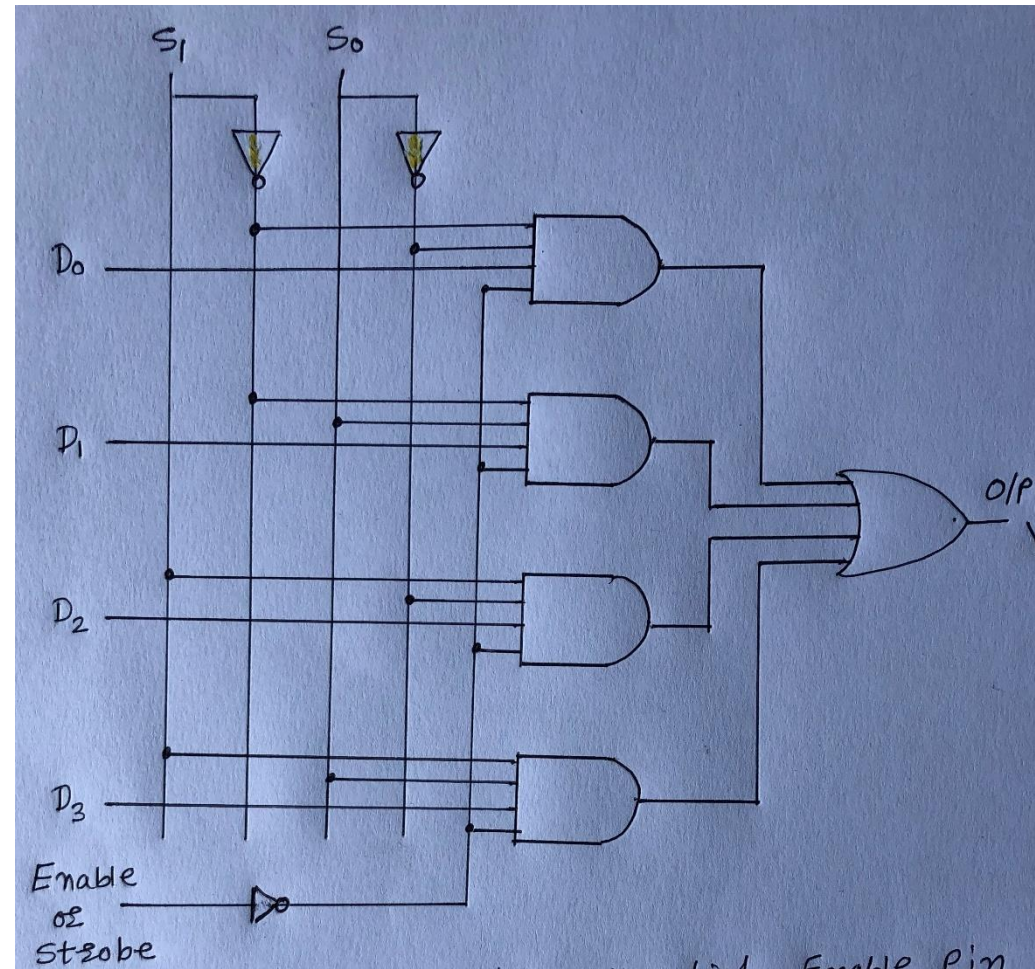
**As this is AND-OR (SOP) circuit, all AND, OR gates can be replaced by NAND gates.**

## What is the function of an enable or strobe input in a multiplexer chip?

Explanation: **Enable or strobe input** is used to active the **chip**, when **enable** is high the **chip** works (ACTIVE), when **enable** is low the **chip** does not work. Enable pin can be used for cascading of ICs.



**If Enable or strobe input is active low. Connect it to Ground to enable (active) the chip. If it is connected to high (Vcc) the **chip** will be disabled.**

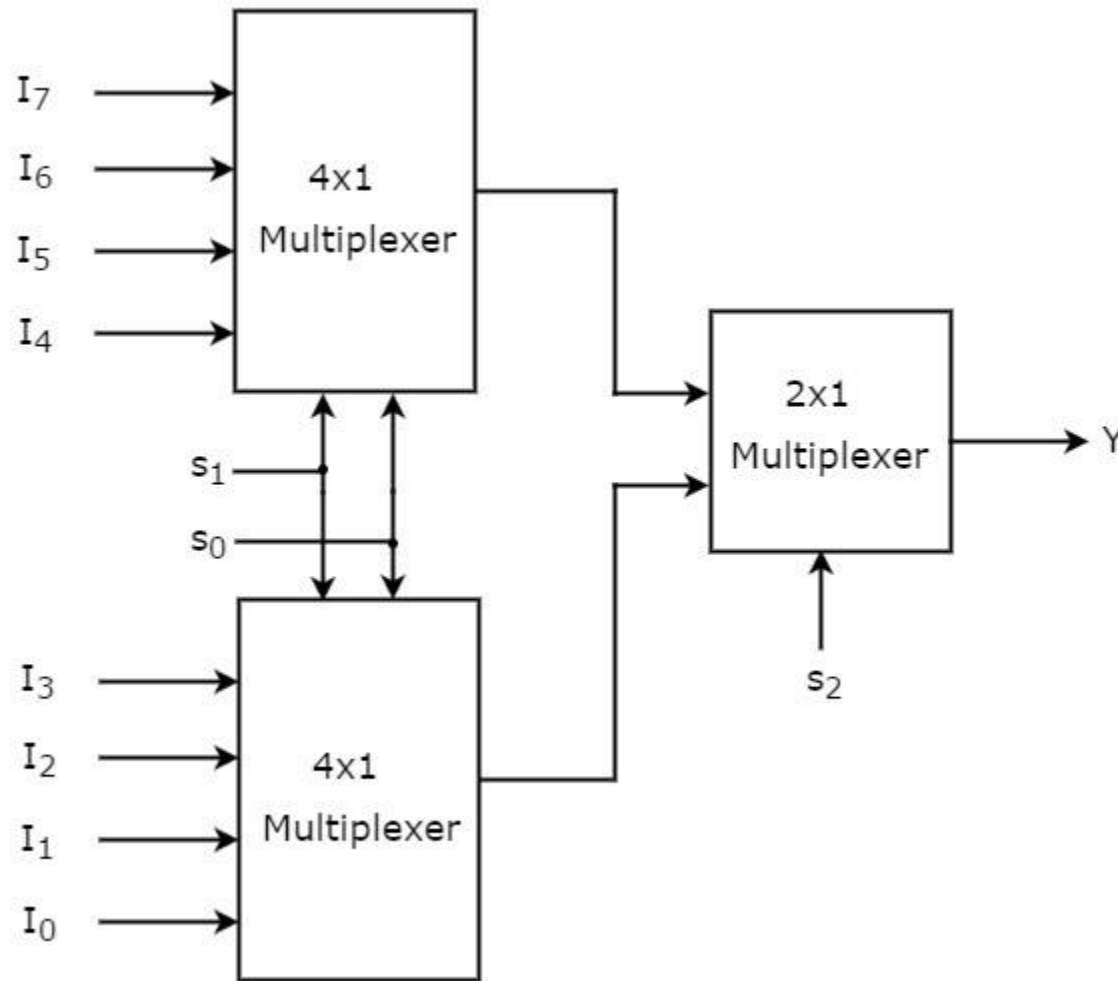


# Truth Table of 8 : 1 Multiplexer

| Selection Inputs |                |                | Output         |
|------------------|----------------|----------------|----------------|
| S <sub>2</sub>   | S <sub>1</sub> | S <sub>0</sub> | Y              |
| 0                | 0              | 0              | I <sub>0</sub> |
| 0                | 0              | 1              | I <sub>1</sub> |
| 0                | 1              | 0              | I <sub>2</sub> |
| 0                | 1              | 1              | I <sub>3</sub> |
| 1                | 0              | 0              | I <sub>4</sub> |
| 1                | 0              | 1              | I <sub>5</sub> |
| 1                | 1              | 0              | I <sub>6</sub> |
| 1                | 1              | 1              | I <sub>7</sub> |

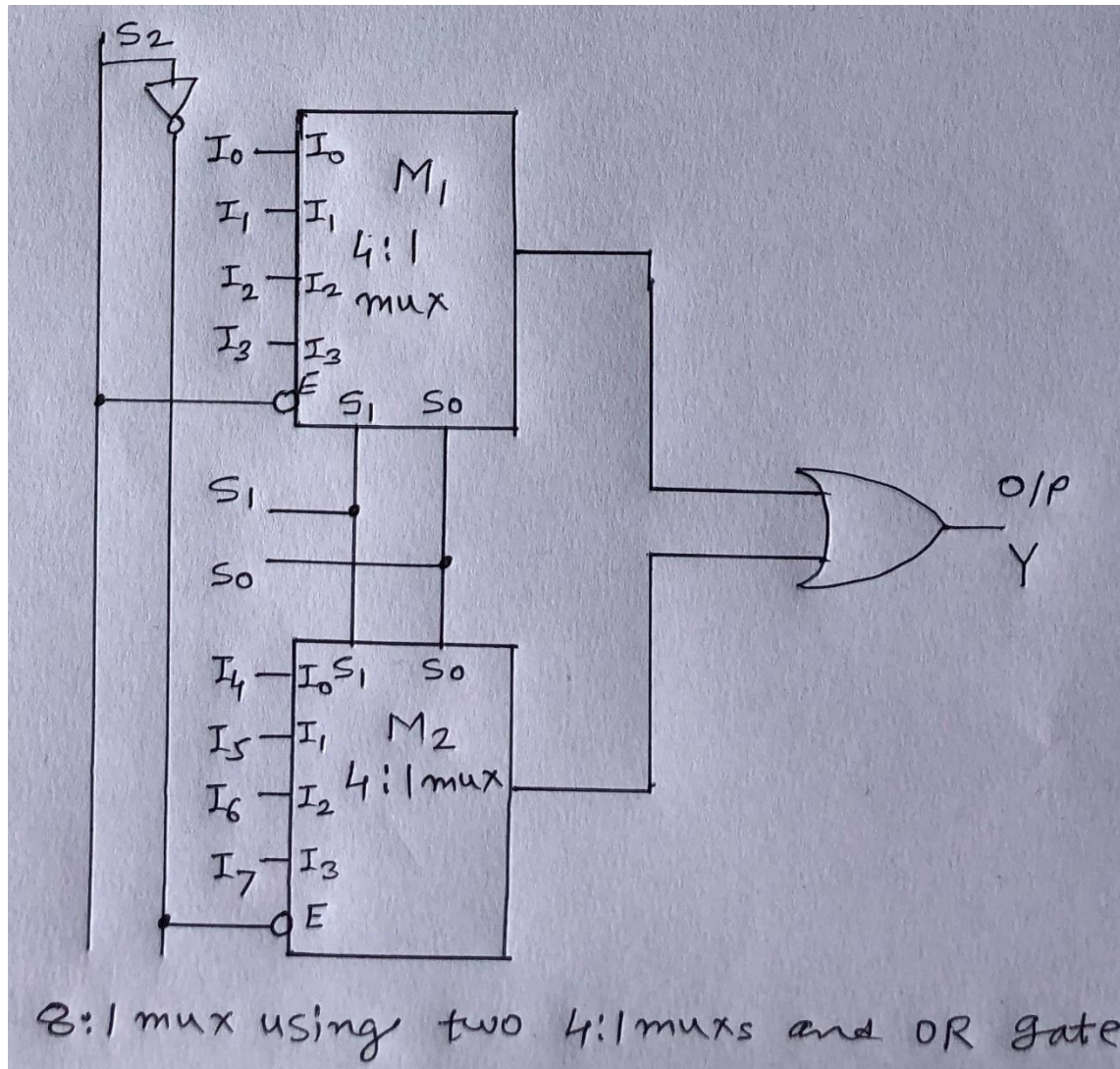
8x1 Multiplexer has eight data inputs I<sub>7</sub> to I<sub>0</sub>, three selection lines S<sub>2</sub>, S<sub>1</sub> & S<sub>0</sub> and one output Y. The **Truth table** of 8 : 1 Multiplexer is shown in adjacent table.

# Cascading of Multiplexer or Multiplexer Tree



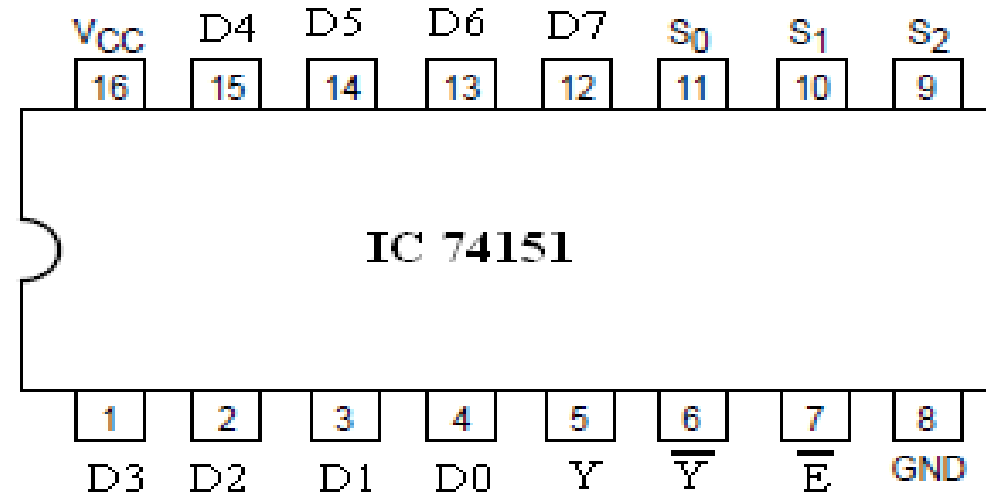
8 : 1 multiplexer using two 4 : 1 mux and one 2 : 1 mux. If Enable pin is active low then connect to ground and if Enable pin is active high then connect to  $V_{cc}$ .

# Cascading of Multiplexer or Multiplexer Tree



8 : 1 multiplexer using two 4 : 1 mux and one OR gate.

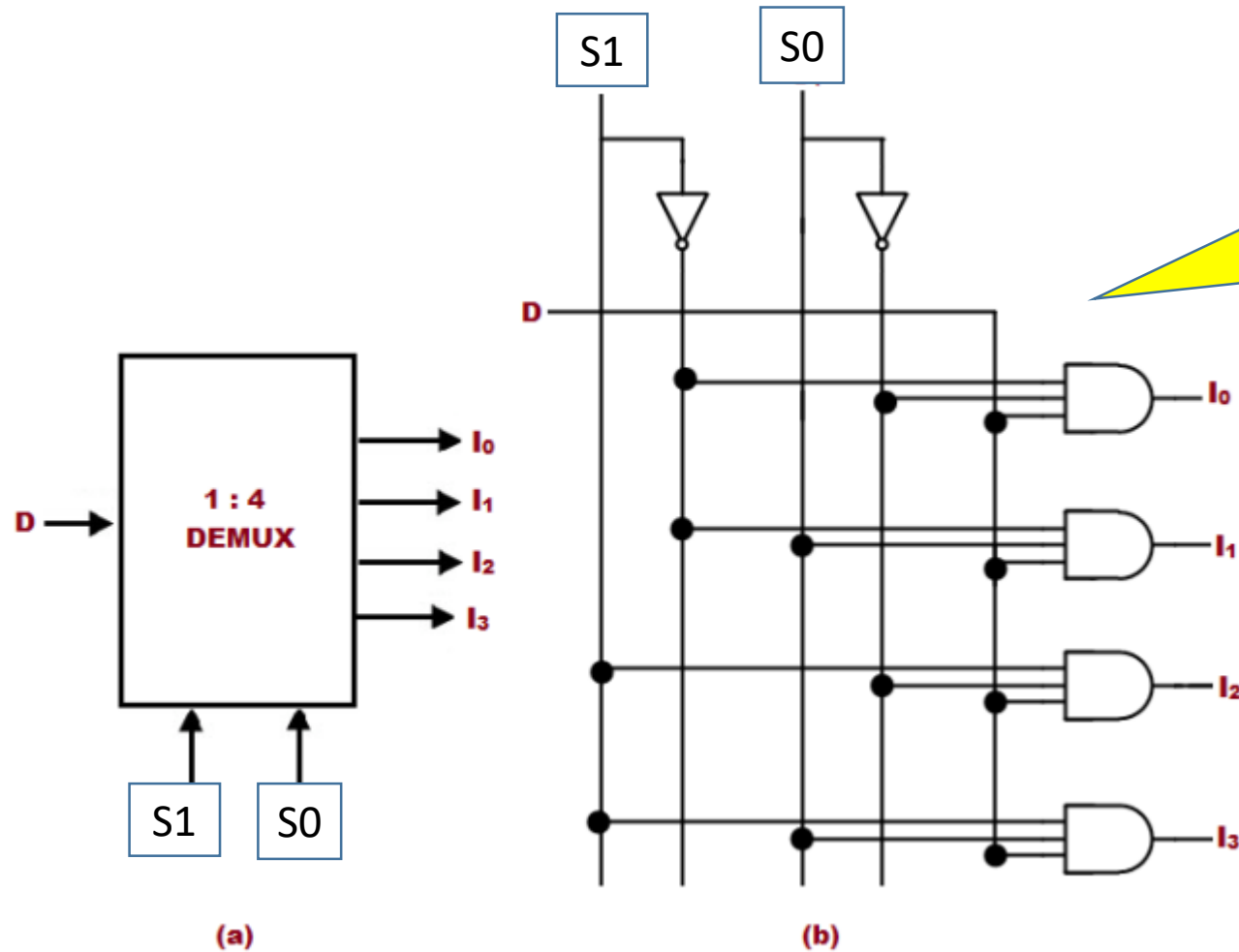
# IC 74151



IC 74151 is 8:1 multiplexer IC. It has three select pins. It has active low enable input pin. The output is available in true as well as complementary form.

# Demultiplexer

Demultiplexer is the combinational circuit which has only one input but many outputs. A **demultiplexer** of  $2^N$  outputs has N select lines, which are used to select which output line to send the input. A **demultiplexer** is also called a data distributor.



**Demux with active high output:**  
The output on selected line will be high and on remaining output line it will be low.

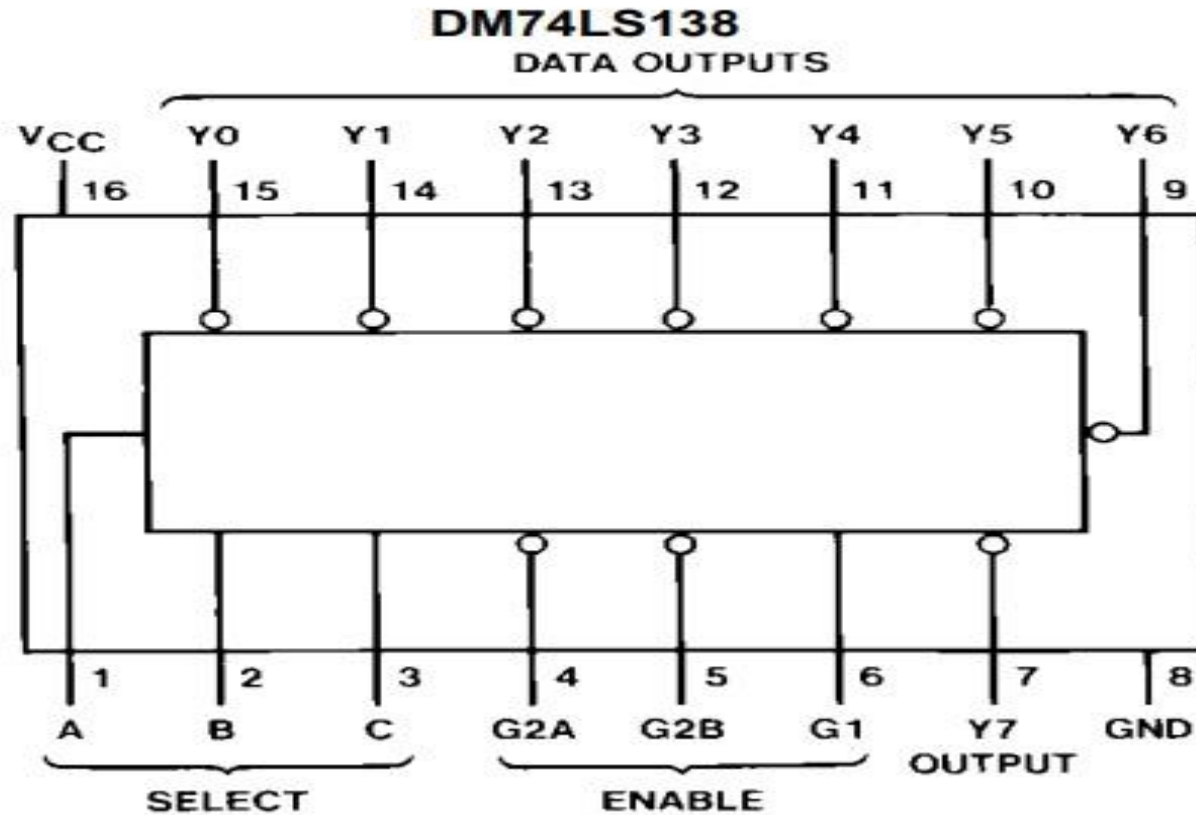
**Demux with active low output:**  
The output on selected line will be low and on remaining output line it will be high. For active low demux use NAND gates instead of AND.

Enable pin can be added just like mux

# Truth table of 1 : 4 De-mux with active high enable pin

| Inputs |    |    | Outputs  |          |          |          |
|--------|----|----|----------|----------|----------|----------|
| E      | S1 | S0 | I3       | I2       | I1       | I0       |
| 0      | X  | X  | 0        | 0        | 0        | 0        |
| 1      | 0  | 0  | 0        | 0        | 0        | <b>D</b> |
| 1      | 0  | 1  | 0        | 0        | <b>D</b> | 0        |
| 1      | 1  | 0  | 0        | <b>D</b> | 0        | 0        |
| 1      | 1  | 1  | <b>D</b> | 0        | 0        | 0        |

# IC 74138



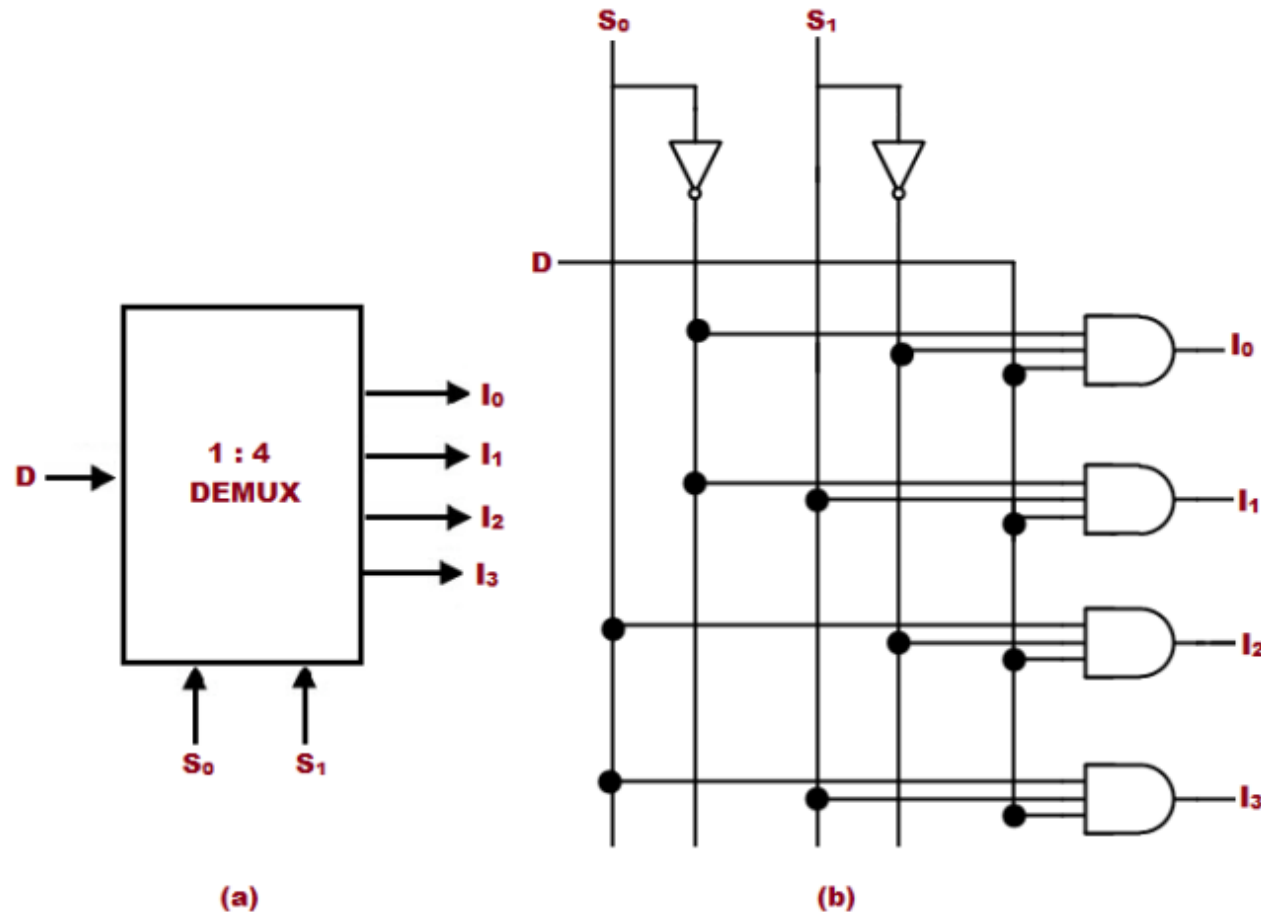
IC 74138 is 1 : 8 demux. The outputs are active low. It has three chip enable pins.

| INPUTS           |                  |       |       |       |       | OUTPUTS |       |       |       |       |       |       |       |
|------------------|------------------|-------|-------|-------|-------|---------|-------|-------|-------|-------|-------|-------|-------|
| $\overline{E_1}$ | $\overline{E_2}$ | $E_3$ | $A_0$ | $A_1$ | $A_2$ | $Y_0$   | $Y_1$ | $Y_2$ | $Y_3$ | $Y_4$ | $Y_5$ | $Y_6$ | $Y_7$ |
| H                | X                | X     | X     | X     | X     | H       | H     | H     | H     | H     | H     | H     | H     |
| X                | H                | X     | X     | X     | X     | H       | H     | H     | H     | H     | H     | H     | H     |
| X                | X                | L     | X     | X     | X     | H       | H     | H     | H     | H     | H     | H     | H     |
| L                | L                | H     | L     | L     | L     | L       | H     | H     | H     | H     | H     | H     | H     |
| L                | L                | H     | H     | L     | L     | H       | L     | H     | H     | H     | H     | H     | H     |
| L                | L                | H     | L     | H     | L     | H       | H     | L     | H     | H     | H     | H     | H     |
| L                | L                | H     | H     | H     | L     | H       | H     | H     | L     | H     | H     | H     | H     |
| L                | L                | H     | L     | L     | H     | H       | H     | H     | H     | L     | H     | H     | H     |
| L                | L                | H     | H     | L     | H     | H       | H     | H     | H     | H     | L     | H     | H     |
| L                | L                | H     | L     | H     | H     | H       | H     | H     | H     | H     | H     | L     | H     |
| L                | L                | H     | H     | H     | H     | H       | H     | H     | H     | H     | H     | H     | L     |

**H - High, L - Low, X - Don't Care**

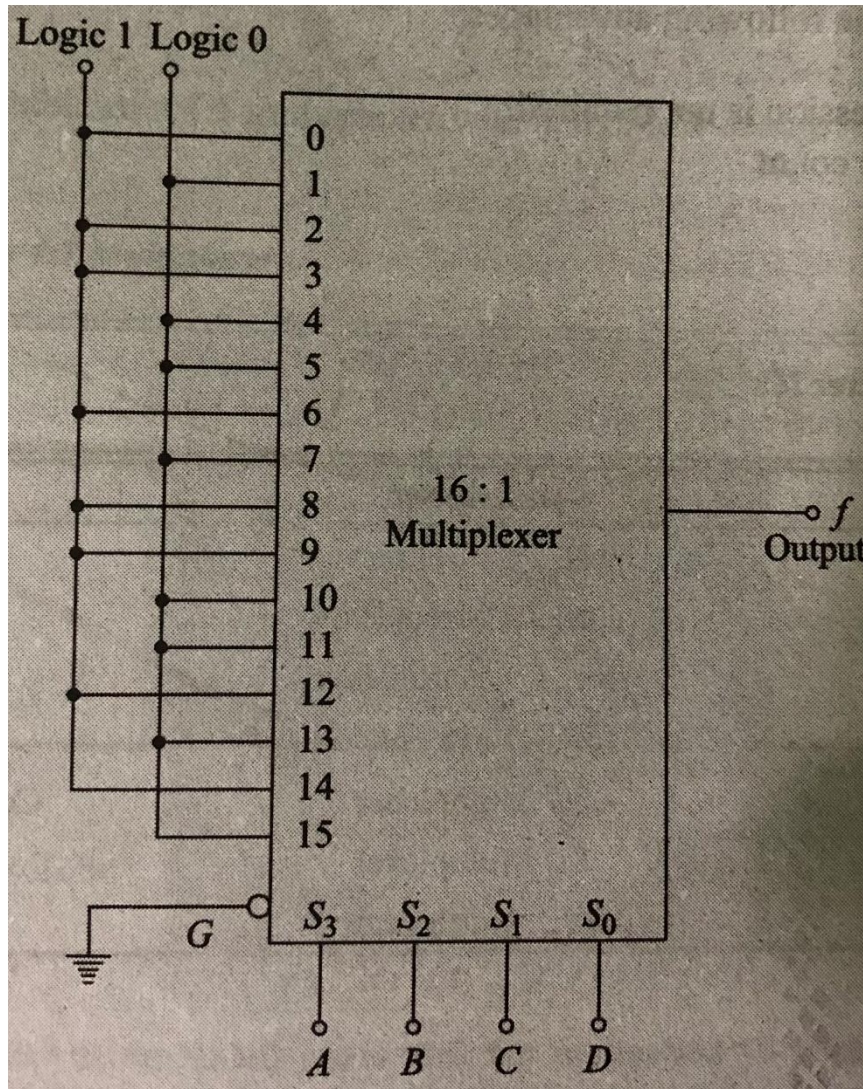
### IC 74138 Truth Table

# Demux as a Decoder



If data input pin is connected to high level ( $V_{cc}$ ), it works like decoder. Adjacent circuit will work like 2 : 4 decoder (Two lines to four lines decoder). The two bit binary number applied to select lines will be decoded. Only one output will become high.

Implement the expression using a multiplexer  
 $f(A, B, C, D) = \sum m(0, 2, 3, 6, 8, 9, 12, 14)$



**For min terms output is high,  
hence connect to VCC.  
For max terms output is low,  
hence connect to ground.**

**Connect chip enable pin to  
ground if it is active low.  
A, B, C, D connect to select  
pins S<sub>3</sub>, S<sub>2</sub>, S<sub>1</sub>, S<sub>0</sub> respectively.**

Implement the expression using a 8 : 1 multiplexer  
 $f(A, B, C, D) = \sum m(2, 4, 6, 7, 9, 10, 11, 12, 15)$

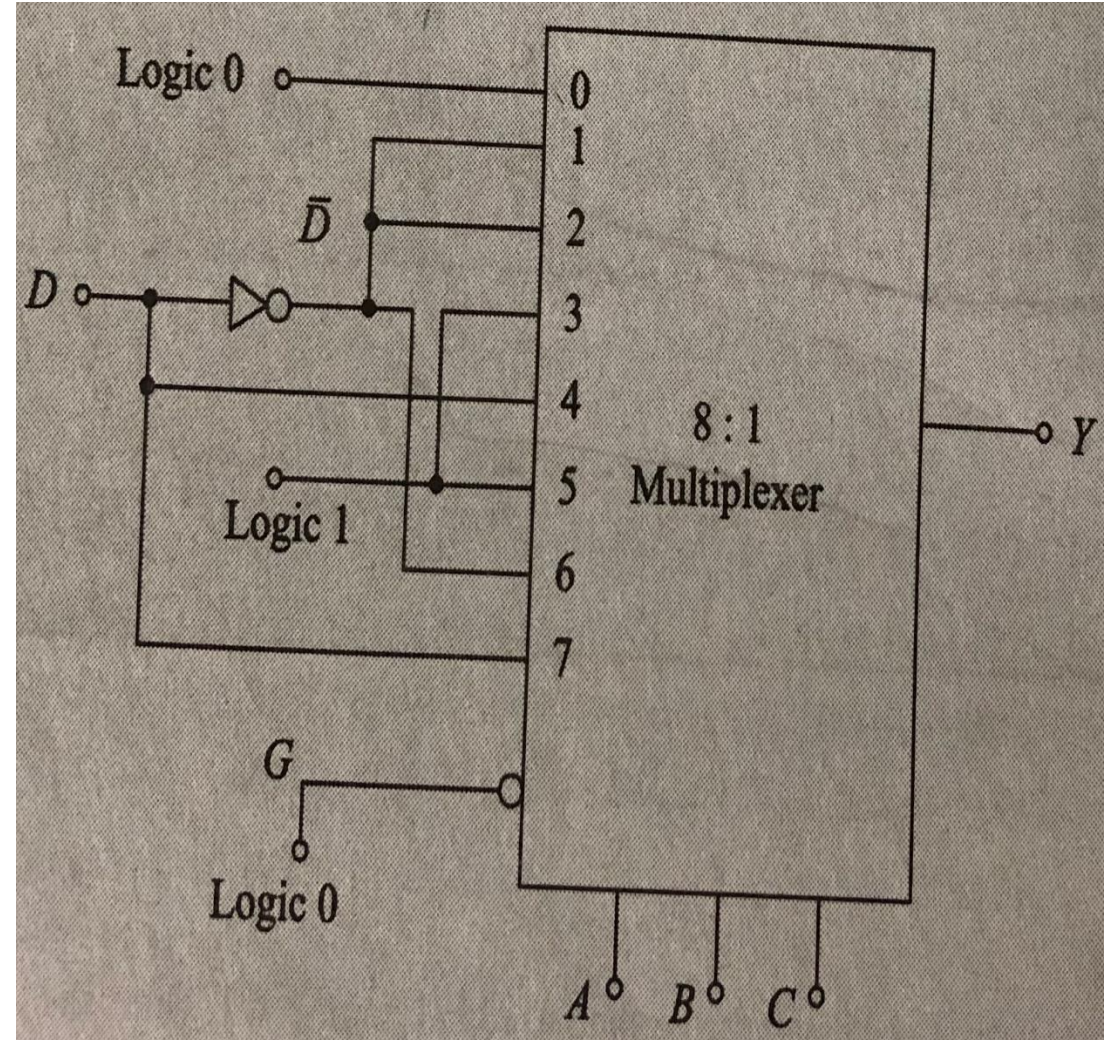
| Inputs |   |   |   | Output |
|--------|---|---|---|--------|
| A      | B | C | D | Y      |
| 0      | 0 | 0 | 0 | 0      |
| 0      | 0 | 0 | 1 | 0      |
| 0      | 0 | 1 | 0 | 1      |
| 0      | 0 | 1 | 1 | 0      |
| 0      | 1 | 0 | 0 | 1      |
| 0      | 1 | 0 | 1 | 0      |
| 0      | 1 | 1 | 0 | 1      |
| 0      | 1 | 1 | 1 | 1      |
| 1      | 0 | 0 | 0 | 0      |
| 1      | 0 | 0 | 1 | 1      |
| 1      | 0 | 1 | 0 | 1      |
| 1      | 0 | 1 | 1 | 1      |
| 1      | 1 | 0 | 0 | 1      |
| 1      | 1 | 0 | 1 | 0      |
| 1      | 1 | 1 | 0 | 0      |
| 1      | 1 | 1 | 1 | 1      |

Divide the TT in a group of two and write output in terms of LSB.

As we want to use 8 : 1 mux, write the TT using 3 variables in which write output in terms of remaining variable, 0 and 1.

| Inputs |   |   | Output    |
|--------|---|---|-----------|
| A      | B | C | Y         |
| 0      | 0 | 0 | 0         |
| 0      | 0 | 1 | $\bar{D}$ |
| 0      | 1 | 0 | $\bar{D}$ |
| 0      | 1 | 1 | 1         |
| 1      | 0 | 0 | D         |
| 1      | 0 | 1 | 1         |
| 1      | 1 | 0 | $\bar{D}$ |
| 1      | 1 | 1 | D         |

| Inputs |   |   | Output    |
|--------|---|---|-----------|
| A      | B | C |           |
| 0      | 0 | 0 | 0         |
| 0      | 0 | 1 | $\bar{D}$ |
| 0      | 1 | 0 | $\bar{D}$ |
| 0      | 1 | 1 | 1         |
| 1      | 0 | 0 | D         |
| 1      | 0 | 1 | 1         |
| 1      | 1 | 0 | $\bar{D}$ |
| 1      | 1 | 1 | D         |



Implement the expression using a 4 : 1 multiplexer and few gates  
 $Y = f(A, B, C, D) = \sum m(1, 4, 5, 7, 9, 10, 11, 12, 15)$

- Divide the TT in a group of four and write output in terms of remaining two variables.

**Using one mux only one function can be realized.**

| Inputs |   |   |   | Output |
|--------|---|---|---|--------|
| A      | B | C | D | Y      |
| 0      | 0 | 0 | 0 | 0      |
| 0      | 0 | 0 | 1 | 1      |
| 0      | 0 | 1 | 0 | 0      |
| 0      | 0 | 1 | 1 | 0      |
| 0      | 1 | 0 | 0 | 1      |
| 0      | 1 | 0 | 1 | 1      |
| 0      | 1 | 1 | 0 | 0      |
| 0      | 1 | 1 | 1 | 1      |
| 1      | 0 | 0 | 0 | 0      |
| 1      | 0 | 0 | 1 | 1      |
| 1      | 0 | 1 | 0 | 1      |
| 1      | 0 | 1 | 1 | 1      |
| 1      | 1 | 0 | 0 | 1      |
| 1      | 1 | 0 | 1 | 0      |
| 1      | 1 | 1 | 0 | 0      |
| 1      | 1 | 1 | 1 | 1      |

|           | $\bar{C}$ | $C$ |
|-----------|-----------|-----|
| $\bar{D}$ | 0         | 0   |
| $D$       | 1         | 0   |

When  $AB = 00$   
 $I_0 = \bar{C}D$

|           | $\bar{C}$ | $C$ |
|-----------|-----------|-----|
| $\bar{D}$ | 0         | 1   |
| $D$       | 1         | 1   |

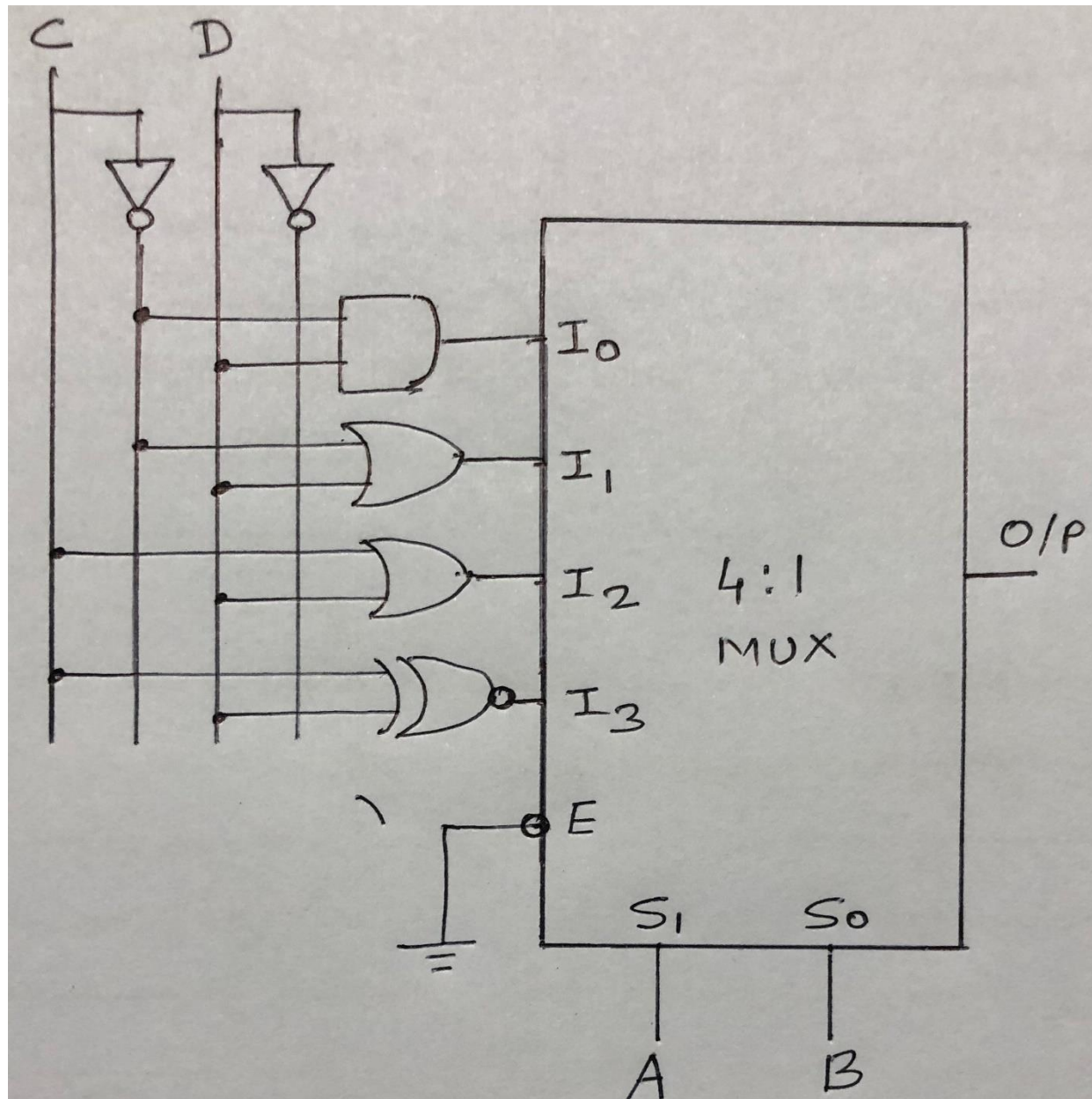
When  $AB = 10$   
 $I_2 = C + D$

|           | $\bar{C}$ | $C$ |
|-----------|-----------|-----|
| $\bar{D}$ | 1         | 0   |
| $D$       | 1         | 1   |

When  $AB = 01$   
 $I_1 = \bar{C} + D$

|           | $\bar{C}$ | $C$ |
|-----------|-----------|-----|
| $\bar{D}$ | 1         | 0   |
| $D$       | 0         | 1   |

When  $AB = 11$   
 $I_3 = \bar{C}\bar{D} + CD$   
 $= C \odot D$

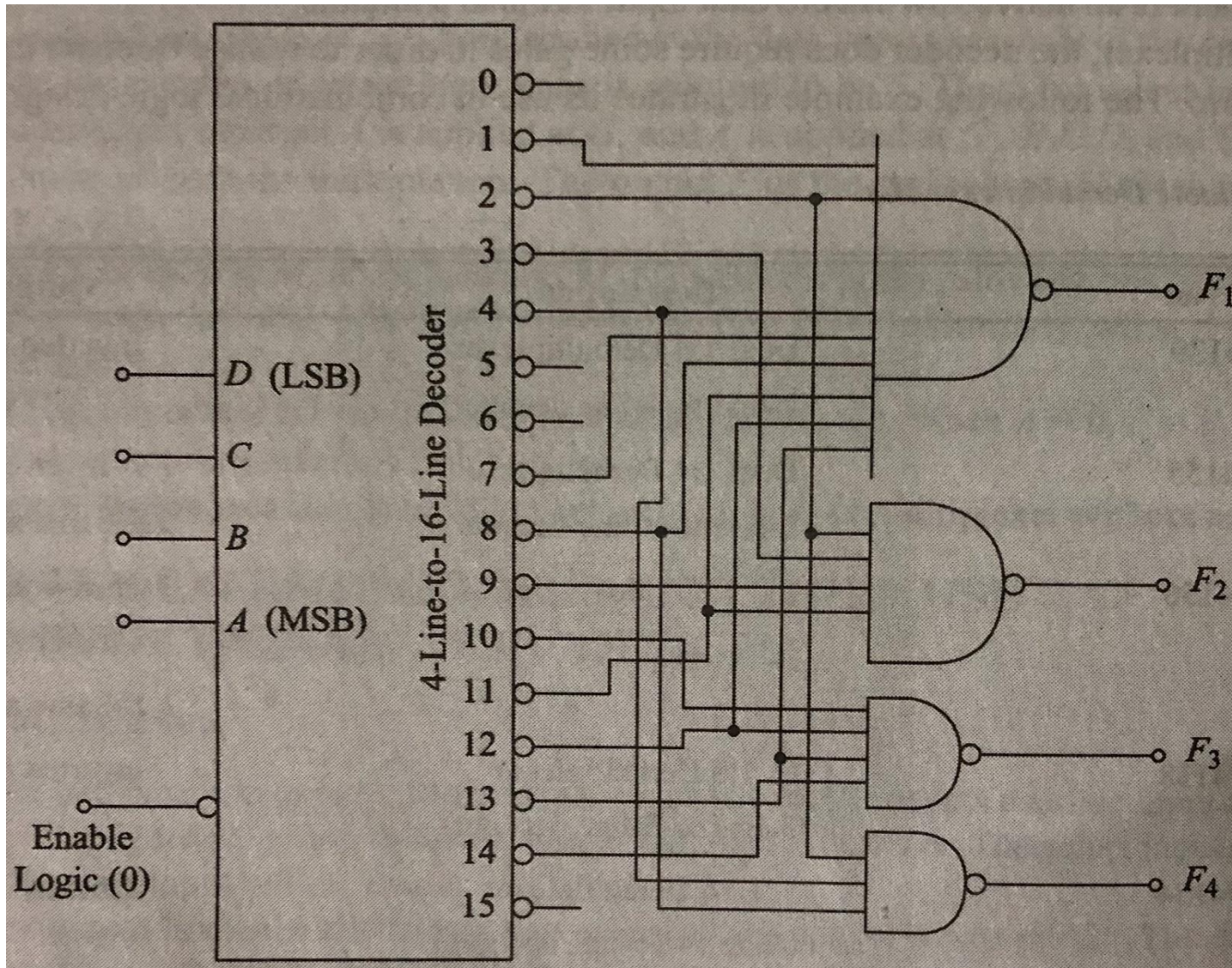


# Implementation of combinational circuit using decoder or demultiplexer

- Implement the following multi-output combinational logic circuit using a 4 lines to 16 lines decoder
- $F1 = (A, B, C, D) = \sum m(1, 2, 4, 7, 8, 11, 12, 13)$
- $F2 = (A, B, C, D) = \sum m(2, 3, 9, 11)$
- $F3 = (A, B, C, D) = \sum m(10, 12, 13, 14)$
- $F4 = (A, B, C, D) = \sum m(2, 4, 8)$

**HW: Implement the full adder circuit using a 3 lines to 8 lines decoder.**

## Implementation of combinational circuit using decoder or demultiplexer



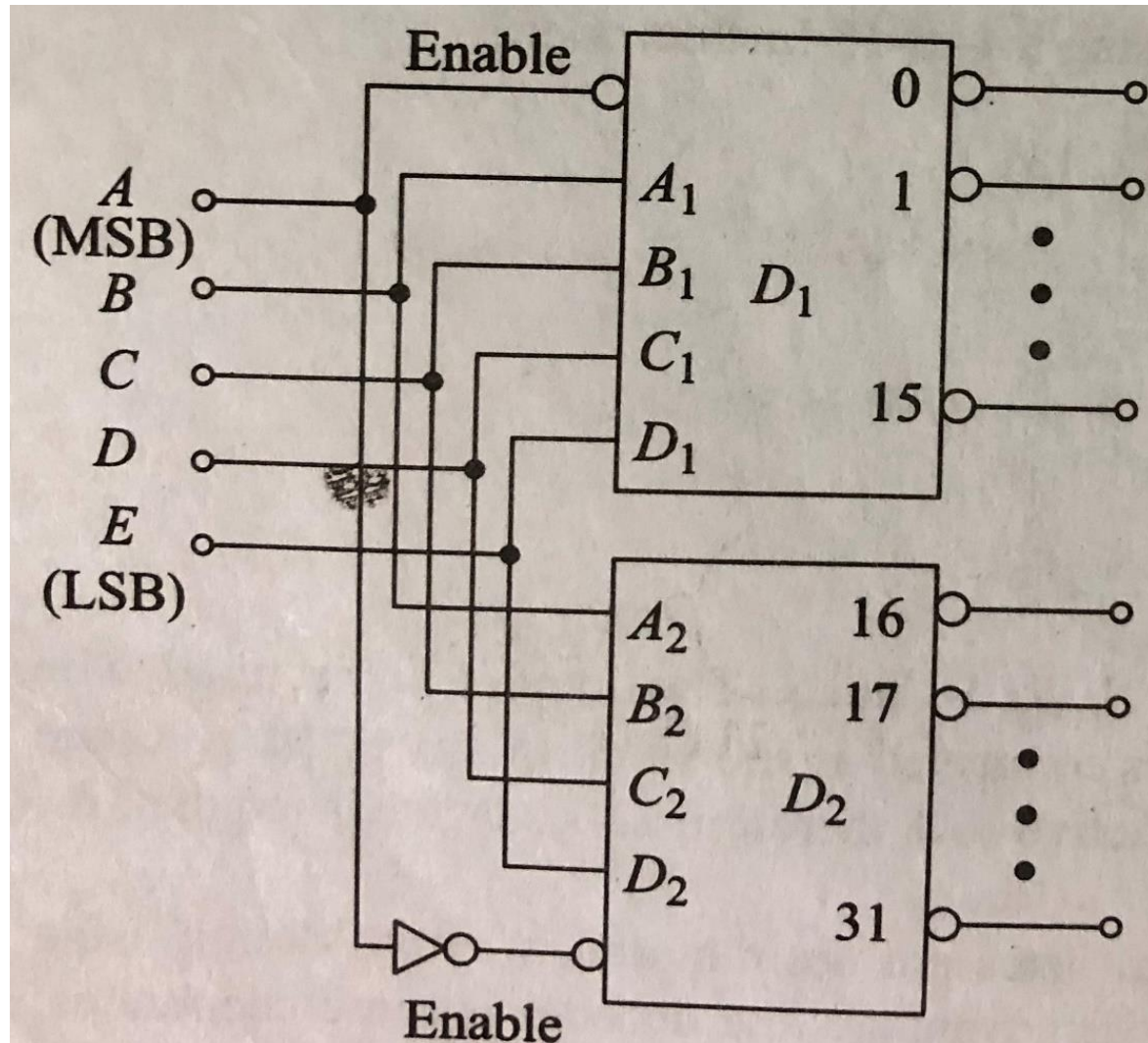
If decoder has active low outputs then for SOP equation use NAND gates.

If decoder has active high outputs then for SOP equation use OR gates.

Using one demux more than one function can be realized.

# Demultiplexer Tree

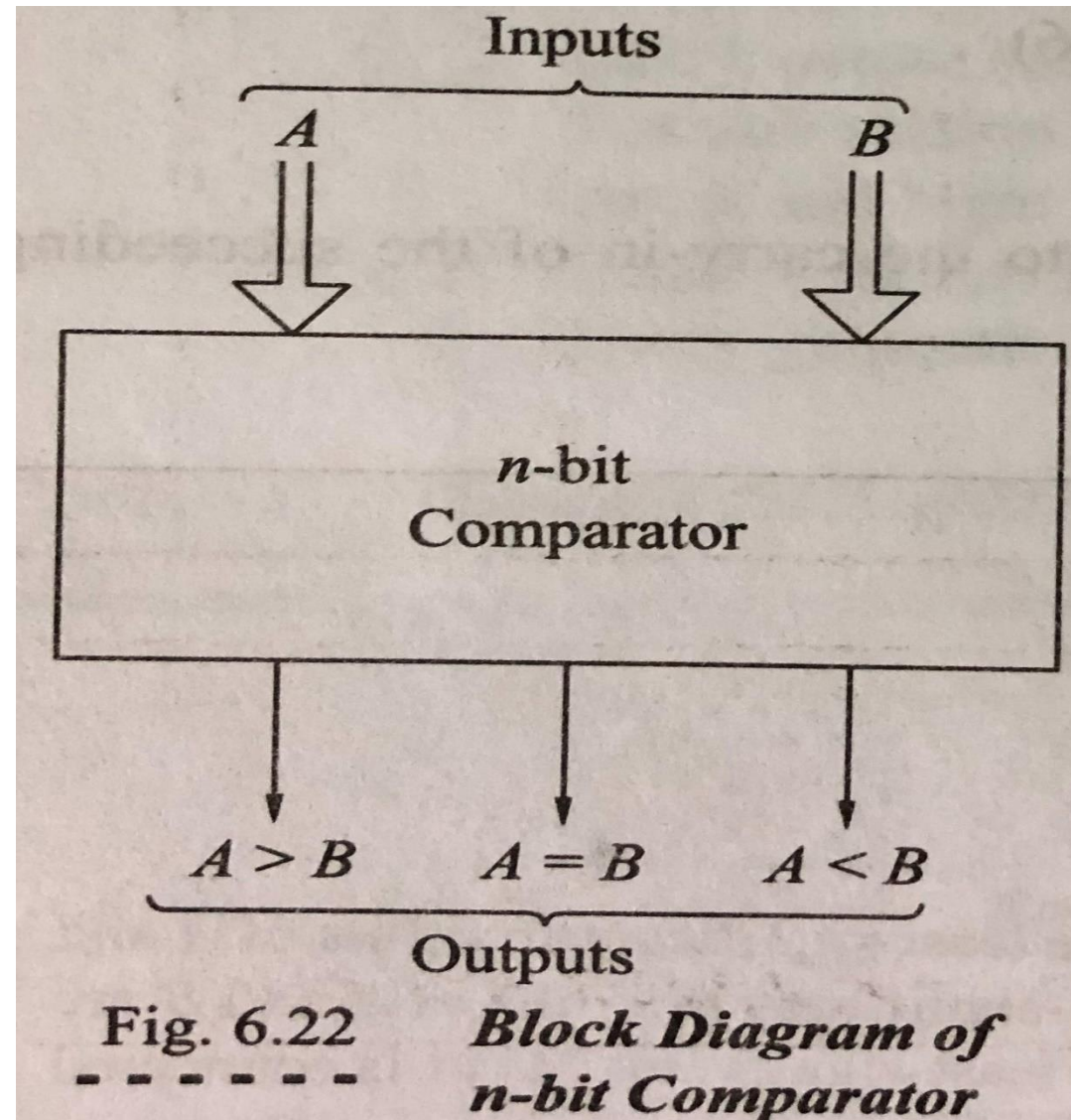
Implement 5-lines to 32-lines decoder using two 4-lines to 16-lines decoders



**When  $A$  is zero, decoder  $D_1$  is enabled and one output line out of 0 to 15 will become zero.**

**When  $A$  is one, decoder  $D_2$  is enabled and one output line out of 16 to 31 will become zero.**

# Digital Comparator



# Truth Table of 2-bit comparator

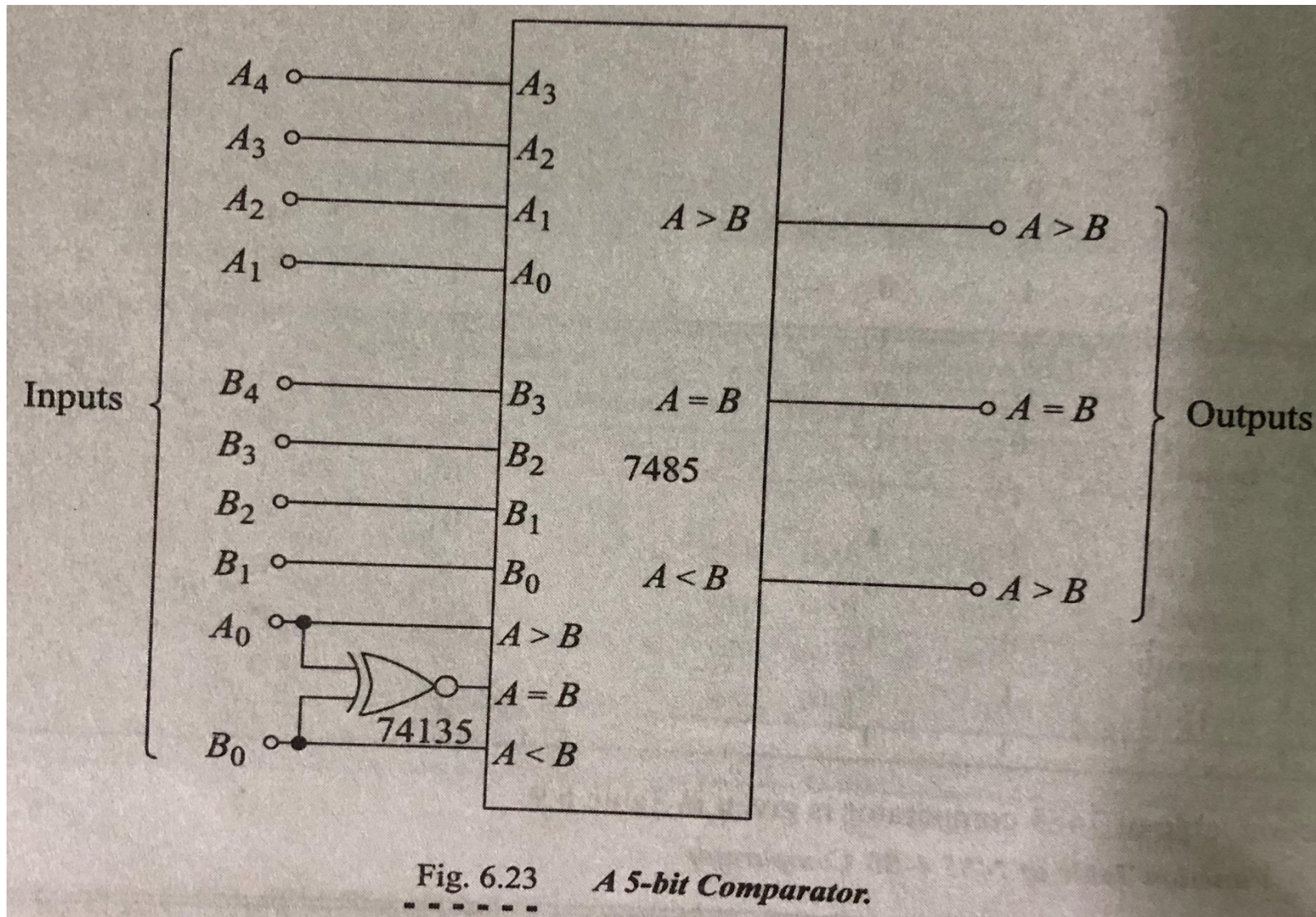
| Inputs |       |       |       | Outputs |         |         |
|--------|-------|-------|-------|---------|---------|---------|
| $A_1$  | $A_0$ | $B_1$ | $B_0$ | $A > B$ | $A = B$ | $A < B$ |
| 0      | 0     | 0     | 0     | 0       | 1       | 0       |
| 0      | 0     | 0     | 1     | 0       | 0       | 1       |
| 0      | 0     | 1     | 0     | 0       | 0       | 1       |
| 0      | 0     | 1     | 1     | 0       | 0       | 1       |
| 0      | 1     | 0     | 0     | 1       | 0       | 0       |
| 0      | 1     | 0     | 1     | 0       | 1       | 0       |
| 0      | 1     | 1     | 0     | 0       | 0       | 1       |
| 0      | 1     | 1     | 1     | 0       | 0       | 1       |
| 1      | 0     | 0     | 0     | 1       | 0       | 0       |
| 1      | 0     | 0     | 1     | 1       | 0       | 0       |
| 1      | 0     | 1     | 0     | 0       | 1       | 0       |
| 1      | 0     | 1     | 1     | 0       | 0       | 1       |
| 1      | 1     | 0     | 0     | 1       | 0       | 0       |
| 1      | 1     | 0     | 1     | 1       | 0       | 0       |
| 1      | 1     | 1     | 0     | 1       | 0       | 0       |
| 1      | 1     | 1     | 1     | 0       | 1       | 0       |

# 4 Bit comparator IC 7485

Table 6.9 *Function Table of 7485 4-Bit Comparator*

| Comparing inputs<br>$A, B$ | Cascading inputs |         |         | Outputs |         |         |
|----------------------------|------------------|---------|---------|---------|---------|---------|
|                            | $A > B$          | $A = B$ | $A < B$ | $A > B$ | $A = B$ | $A < B$ |
| $A > B$                    | X                | X       | X       | 1       | 0       | 0       |
| $A = B$                    | 1                | 0       | 0       | 1       | 0       | 0       |
|                            | X                | 1       | X       | 0       | 1       | 0       |
|                            | 0                | 0       | 1       | 0       | 0       | 1       |
|                            | 0                | 0       | 0       | 1       | 0       | 1       |
|                            | 1                | 0       | 1       | 0       | 0       | 0       |
|                            | X                | X       | X       | 0       | 0       | 1       |
| $A < B$                    | X                | X       | X       | 0       | 0       | 1       |

# 5 Bit comparator using IC 7485



# Operation of 5 bit comparator

Ex  $\rightarrow A = B$

Let  $A = 20 \rightarrow$

| $A_4$ | $A_3$ | $A_2$ | $A_1$ | $A_0$ |
|-------|-------|-------|-------|-------|
| 1     | 0     | 1     | 0     | 0     |

$B = 20 \rightarrow$

| $B_4$ | $B_3$ | $B_2$ | $B_1$ | $B_0$ |
|-------|-------|-------|-------|-------|
| 1     | 0     | 1     | 0     | 0     |

EXNOR gate o/p is 1

$\therefore$  Cascading i/p are

| $A > B$ | $A = B$ | $A < B$ |
|---------|---------|---------|
| 0       | 1       | 0       |

$\therefore$  From function table o/p will be

| $A > B$ | $A = B$ | $A < B$ |
|---------|---------|---------|
| 0       | 1       | 0       |

$$EX \rightarrow A = B = 21$$

$A_4 A_3 A_2 A_1 A_0$   
 $1 \ 0 \ 1 \ 0 \ 1$

$B_4 B_3 B_2 B_1 B_0$   
 $1 \ 0 \ 1 \ 0 \ 1$

EX NOR gate o/p is 1

$\therefore$  cascading i/p are

|         |         |         |
|---------|---------|---------|
| $A > B$ | $A = B$ | $A < B$ |
| 1       | 1       | 1       |

$\therefore$  From function table o/p will be

|         |         |         |
|---------|---------|---------|
| $A > B$ | $A = B$ | $A < B$ |
| 0       | 1       | 0       |

EX  $\rightarrow A > B$

$A = 21$

$B = 20$

$A_4 A_3 A_2 A_1 A_0$

1 0 1 0 1

$B_4 B_3 B_2 B_1 B_0$

1 0 1 0 0

EXNOR gate o/p is 0

$\therefore$  Cascading i/p's are

$A > B$

$A = B$

$A < B$

1

0

0

$\therefore$  From function table o/p will be

$A > B$

$A = B$

$A < B$

1

0

0

Ex  $A = 18$  ,  $B = 23$

$$\begin{array}{cccccc} A_4 & A_3 & A_2 & A_1 & A_0 \\ 1 & 0 & 0 & 1 & 0 \end{array}$$

$$\begin{array}{ccccc} B_4 & B_3 & B_2 & B_1 & B_0 \\ 1 & 0 & 1 & 1 & 1 \end{array}$$

$$\begin{array}{ccccc} A_4 & A_3 & A_2 & A_1 & B_4 & B_3 & B_2 & B_1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \end{array}$$

$$A < B$$

EXNOR gate o/p is 0

∴ Cascading i/p's are

|         |         |         |
|---------|---------|---------|
| $A > B$ | $A = B$ | $A < B$ |
| 0       | 1       | 0       |

∴ From function table when  $A < B$

and cascading i/p  $\times \times \times$  the o/p will be

|         |         |         |
|---------|---------|---------|
| $A > B$ | $A = B$ | $A < B$ |
| 0       | 0       | 1       |